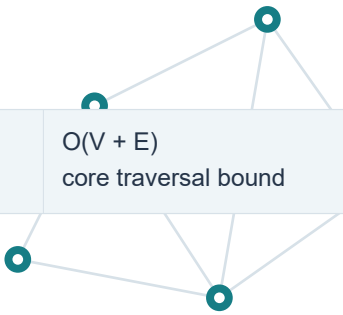
COMPUTER SCIENCE / DATA STRUCTURES & ALGORITHMS

Graph Data Structures & Algorithms

A structured, web-ready learning module with graph anatomy, representations, traversal traces, and an organized source reference.



22	73	O(V + E)
source topics	reference pages	core traversal bound

Prepared from the supplied merged lecture notes

Concept guide and navigation: newly organized. Original C++ guides and diagrams: preserved in the reference section.

START HERE

How to use this guide

The first 11 pages provide a consistent conceptual layer. The complete source PDF follows as a 73-page reference appendix.

PART	WHAT IT COVERS
01 Graph anatomy	Vertices, edges, direction, weight, connectivity, cycles, and a repeatable diagram-reading checklist.
02 Representations	Adjacency list and matrix, a JSON data model, and minimal traversal code.
03 Algorithm families	Traversal, connectivity, DAG ordering, shortest paths, spanning trees, and critical structure.
04 Reference appendix	All original pages retained; use the PDF bookmarks and page map on page 11.

Reading a diagram consistently

1. Identify	2. Record	3. Trace	4. Encode
Directed or undirected; weighted or unweighted; connected or not; cyclic or acyclic.	Every vertex and edge. Preserve arrow direction, edge weight, labels, and special marks.	Name the start state, queue/stack or priority structure, visits, relaxations, and output.	Choose list for sparse traversal, matrix for dense lookup, and keep metadata separate from topology.

The source is a collection of focused C++ study guides rather than one originally typeset book. Its text and diagrams are preserved in the appendix; this opening section standardizes their concepts and web-facing examples.

01 / STRUCTURAL ANATOMY

A graph is a set of entities and relationships

Use G = (V, E). A node is a vertex in V; an edge in E states which vertices are connected and, when applicable, in which direction and at what cost.

PROPERTY	INTERPRETATION	WHAT TO CAPTURE
Direction	Undirected edges permit travel both ways; directed edges permit travel only along the arrow.	Record each arc once for a digraph. Record each undirected edge once canonically, even if adjacency lists show it twice.
Weight	A numeric edge cost, distance, time, or capacity.	Include the value and meaning; absence means unweighted, not weight zero.
Cycle structure	A cycle returns to its starting vertex. A DAG is a directed acyclic graph.	A tree is connected and acyclic (undirected); a forest is a disjoint union of trees.
Connectivity	Connected components partition an undirected graph. SCCs do so for mutual reachability in a digraph.	List isolated vertices and components; for directed graphs distinguish reachability from strong connectivity.
Bipartite	Vertices split into two sets and every edge crosses the partition.	Record the two partitions; an undirected graph is bipartite iff it has no odd cycle.

Inventory checklist

Nodes: stable ID, display label, optional category/state/coordinates. Edges: stable ID when needed, source, target, direction, optional weight/type. Algorithm state: visited, parent, distance, discovery/low time, indegree, or component ID as required by the lesson. Keep algorithm state separate from the underlying graph.

Do not infer a weight, edge, or traversal tie-break from visual proximity. If the source does not specify neighbor order, multiple valid traversal orders may exist.

02 / WEB-READY REPRESENTATION

Store topology separately from the view

The source reference graph has vertices 0–4 and undirected edges (0,1), (1,2), (1,3), (2,3), and (2,4).

Canonical edge inventory

EDGE	DIRECTION	WEIGHT
0 — 1	Undirected / traversable both ways	None
1 — 2	Undirected / traversable both ways	None
1 — 3	Undirected / traversable both ways	None
2 — 3	Undirected / traversable both ways	None
2 — 4	Undirected / traversable both ways	None

JSON: adjacency list and matrix

```
{
  "directed": false,
  "weighted": false,
  "nodes": [{"id":0}, {"id":1}, {"id":2}, {"id":3}, {"id":4}],
  "edges": [[0,1], [1,2], [1,3], [2,3], [2,4]],
  "adjacencyList": {"0":[1], "1":[0,2,3], "2":[1,3,4],
                    "3":[1,2], "4":[2]},
  "adjacencyMatrix": [[0,1,0,0,0], [1,0,1,1,0],
                      [0,1,0,1,1], [0,1,1,0,0], [0,0,1,0,0]]
}
```

Representation trade-off

Adjacency lists use O(V + E) space and enumerate neighbors in O(degree(u)); they suit sparse graphs and BFS/DFS. An adjacency matrix uses O(V²) space and answers edge existence in O(1); it suits dense graphs or matrix-native input such as Number of Provinces.

03 / TRAVERSAL

BFS and DFS share the graph; their frontier differs

For the five-vertex reference graph, start at 0 and examine neighbors in ascending order. This explicit tie-break makes the trace reproducible.

Breadth-first search (FIFO queue)

STEP	REMOVE	NEWLY DISCOVERED	QUEUE AFTER EXPANSION
0	0	1, 2	[1, 2]
1	1	3	[2, 3]
2	2	4	[3, 4]
3	3	—	[4]
4	4	—	[]

Result: 0 → 1 → 2 → 3 → 4. Parent pointers yield one shortest unweighted path from 0 to each reachable vertex. Mark a vertex visited when enqueuing it, so it cannot enter the queue twice.

Depth-first search (recursive call stack or explicit LIFO stack)

Using ascending neighbors, recursive DFS visits 0 → 1 → 2 → 3, backtracks to 2, then visits 4. DFS order depends on neighbor ordering; it does not generally give shortest paths.

```
function bfs(adj, start) {
  const seen = new Set([start]), queue = [start], order = [];
  for (let head = 0; head < queue.length; head++) {
    const u = queue[head]; order.push(u);
    for (const v of adj[u]) if (!seen.has(v)) {
      seen.add(v); queue.push(v);
    }
  }
  return order;
}
```

Both traversals run in O(V + E) with adjacency lists when all components are covered. A single-source call visits only the source component; wrap it in an outer loop to process a disconnected graph.

04 / CONNECTIVITY AND IMPLICIT GRAPHS

Components, grids, and multi-source expansion

Several source sections represent a graph implicitly. A grid cell is a vertex; allowed neighbor offsets define edges. The diagram is still a graph even when no explicit node-link drawing appears.

TOPIC	MODEL / TYPE	FLOW OR OUTPUT
Number of Provinces	Undirected graph encoded by a symmetric adjacency matrix.	Scan vertices; each unseen vertex starts a DFS/BFS and increments the component count. Matrix traversal costs $O(V^2)$.
Number of Islands	Binary grid; land cells connect to allowed adjacent land cells (commonly 4-neighbor).	Scan cells; each unvisited land cell starts one flood traversal and one island count.
Flood Fill	Grid graph restricted to cells matching the starting color.	DFS/BFS recolors each reachable matching cell once. Return immediately if source color equals target color.
Rotting Oranges	Grid graph with all initially rotten cells as simultaneous BFS sources.	Expand one queue layer per minute. Return -1 if any fresh orange remains unreachable; otherwise return elapsed layers.

Traversal blueprint for grid modules

Represent a cell by (row, column); enumerate four offsets (−1,0), (0,1), (1,0), (0,−1), or add diagonals only when the problem specifies them. Check bounds before reading a neighbor. Mark cells at discovery time. For multi-source BFS, enqueue all sources before processing the first layer.

A graph is bipartite exactly when it admits a 2-coloring with every edge crossing colors. A BFS/DFS coloring can also detect odd cycles. The supplied bundle classifies bipartite graphs but does not include a dedicated worked bipartite traversal.

05 / DIRECTED STRUCTURE

DAG ordering and cycle detection

A topological order exists only for a directed acyclic graph. Every edge u → v requires u to appear before v; there may be several valid orders.

Source worked DAG

```
V = {0,1,2,3,4,5}
E = {5->0, 5->2, 4->0, 4->1, 2->3, 3->1}
Kahn trace (one valid neighbor/queue order):
initial zero-indegree queue [4,5]
pop 4; pop 5; pop 0; pop 2; pop 3; pop 1
valid output: [4,5,0,2,3,1]
```

Kahn's algorithm repeatedly removes zero-indegree vertices; if fewer than V vertices are emitted, a directed cycle remains. DFS topological sorting pushes each node on exit and reverses finish order; cycle detection uses three states (unseen, active, complete), with an edge to an active vertex signaling a back edge.

Course Schedule is the same dependency graph: prerequisite b for course a becomes b → a. A valid schedule exists exactly when all vertices can be emitted.

Cycle rules differ by graph type

GRAPH	DFS / BFS SIGNAL	IMPORTANT DETAIL
Undirected	An already visited neighbor other than the current node's parent indicates a cycle.	For multigraphs, identify the parent edge by edge ID; skipping every edge to the parent vertex can miss a parallel-edge cycle.
Directed	An edge to a vertex on the current DFS path (active state) indicates a cycle.	An edge to a completed vertex alone is not evidence of a cycle.

06 / WEIGHTED PATHS

Choose shortest-path algorithms by weight constraints

“Shortest” means minimum total edge weight. In an unweighted graph, BFS minimizes number of edges; weighted graphs require explicit relaxation.

ALGORITHM	GRAPH / TASK	CORE FLOW	TIME
BFS	Unweighted single-source	Queue by hop distance; store parent and distance.	O(V+E)
Dijkstra	Nonnegative weights; single-source	Min-priority queue; relax outgoing edges; ignore stale heap entries.	O((V+E) log V) with binary heap
Bellman–Ford	Negative edges allowed; single-source	Relax all edges V−1 rounds; another improving round signals a reachable negative cycle.	O(VE)
Floyd–Warshall	All-pairs shortest paths	Dynamic programming over allowed intermediate vertices.	$O(V^3)$
K-stop flights	Directed weighted path with edge-count bound	Track cost by number of edges/round; retain the stop constraint in state.	O(K E) layered relaxation

Shortest-path representation

For weighted adjacency lists use entries {to, weight}; preserve direction. Store predecessor separately to reconstruct paths. Distances should have an explicit unreachable sentinel. Never use Dijkstra when negative edge weights are possible. For stop-limited flights, a single best cost per vertex can discard a more useful lower-stop route; the constraint must be represented in the state or relaxation schedule.

In the appendix, flight costs and the Bellman–Ford/Dijkstra diagrams are source examples; inspect their original edge labels before reusing a worked numerical trace.

07 / SPANNING STRUCTURES

A minimum spanning tree connects every vertex cheaply

For a connected, weighted, undirected graph, an MST has V−1 edges, spans all vertices, and contains no cycle. If the graph is disconnected, the result is a minimum spanning forest.

ALGORITHM	GROWTH STRATEGY	SELECTION	TYPICAL COMPLEXITY
Kruskal	Global edge-centric	Sort edges; accept if DSU says endpoints are in different components.	O(E log E)
Prim	One tree grows outward	Repeatedly add the lightest edge crossing from the tree to an unvisited vertex.	O(E log V) with heap/list

Source Kruskal trace

The five-vertex example sorts edges by weights 2, 3, 5, 6, 7, 8, 9. It accepts (0,1), (1,2), (1,4), (0,3), reaching total weight 16 and V−1 = 4 edges. Remaining edges are rejected because they would close a cycle.

Disjoint Set Union (Union-Find)

DSU maintains a partition of vertices into disjoint sets. find(x) returns a representative; union(a,b) merges different sets. Path compression plus union by rank/size gives amortized near-constant operations, commonly written O(α(V)). DSU supports Kruskal and connectivity tracking, but does not itself encode paths or preserve the graph’s full topology.

Visual encoding

Show all candidate edges in a muted tone, accepted MST edges in the accent color, and rejected cycle-forming edges with a distinct dashed or faded treatment. Animate one sorted edge at a time alongside the DSU component partition and running total.

08 / CRITICAL STRUCTURE

Strong connectivity, bridges, and cut vertices

These algorithms identify how directed reachability or undirected connectivity changes when edges or vertices are removed.

CONCEPT	GRAPH TYPE	TEST / METHOD	OUTPUT
SCC (Kosaraju)	Directed	DFS finish-time stack; transpose all edges; DFS in stack order.	Maximal mutually reachable vertex sets.
Bridge	Undirected	For DFS child v of u: low[v] > tin[u].	An edge whose removal disconnects its component.
Articulation point	Undirected	Non-root u: low[v] >= tin[u]; root: more than one DFS-tree child.	A vertex whose removal increases component count.

Tarjan low-link invariant

tin[u] is the DFS discovery time. low[u] is the earliest discovery time reachable from u's DFS subtree using tree edges plus at most one back edge. Update low values after child recursion; treat visited non-parent edges as back edges. Iterate every vertex to handle disconnected undirected graphs.

Caution: parent-by-vertex code in the source is suitable for a simple graph. A multigraph implementation must distinguish the incoming tree edge from a parallel edge using edge IDs.

Web-ready state overlays

Keep topology stable while algorithm state changes: discovery indices on nodes, low-link values in a side panel, DFS tree edges in one style, back edges in another; identify bridge edges and articulation vertices only when their conditions become established. For Kosaraju, show G, G^T, and component coloring as three synchronized panels.

09 / INTERACTIVE BLUEPRINT

Translate a 2D sketch into a teachable web view

Use a layout for the structure and reserve motion for the algorithm state. A faithful diagram is more useful than a decorative 3D extrusion when edge direction and crossings carry meaning.

LAYER	RENDERING GUIDANCE
Topology	SVG for labeled nodes, arrowheads, weights, loops, and crisp edges; Canvas/WebGL for large dense graphs. Keep edge hit areas wider than visible strokes.
Layout	Use a fixed hand-tuned layout for a small worked example. Use layered DAG layout for topological sorting, concentric rings for BFS levels/SCCs, and grid coordinates for matrix problems. Preserve source positions when reconstructing an annotated sketch.
State	Visited nodes, active frontier, parent/tree edges, distances, low-link values, indegrees, and components get separate visual channels. Include a legend and a textual live status.
Motion	Advance one algorithm event at a time; show the queue/stack/heap and state change together. Provide pause, step, reset, and reduced-motion behavior. Never rely on color alone.
Accessibility	Keyboard-operable controls, visible focus, high contrast, non-color labels/patterns, and an adjacent textual edge/traversal listing.

Suggested fixed layout for the five-node example

Place 0 at (70,150), 1 at (190,75), 2 at (320,150), 3 at (250,260), and 4 at (430,210) in a 500×320 viewBox. Draw five undirected edges from the canonical edge list; route (1,2) and (2,3) with enough separation to avoid ambiguous crossings. BFS animation begins at 0, expands neighbors 1, then 2 and 3, then 4; show levels d={0:0,1:1,2:2,3:2,4:3}.

This coordinate set is a new layout proposal for the source graph, not a measurement from the original sketch.

REFERENCE APPENDIX

Source page map

The appendix begins immediately after this 12-page module. Page numbers below refer to original source pages; add 12 to find the combined PDF page.

TOPIC	SOURCE PAGES	COMBINED PDF
Strongly Connected Components / Kosaraju	1–3	13–15
Articulation Points / Tarjan	4–6	16–18
Bridges / Tarjan	7–9	19–21
Cheapest Flights Within K Stops	10–11	22–23
Number of Provinces	12–13	24–25
Kruskal Minimum Spanning Tree	14–17	26–29
Disjoint Set Union / Union-Find	18–20	30–32
Prim Minimum Spanning Tree	21–23	33–35
Bellman–Ford	24–26	36–38
Dijkstra	27–30	39–42
Topological Sort / Kahn	31–33	43–45
Flood Fill	34–36	46–48
Course Schedule	37–39	49–51
Topological Sort / DFS	40–42	52–54
Directed Cycle Detection / DFS	43–45	55–57
Rotting Oranges / Multi-source BFS	46–49	58–61
Number of Islands	50–52	62–64
Undirected Cycle Detection / BFS	53–55	65–67
Undirected Cycle Detection / DFS	56–58	68–70
Depth-First Search	59–62	71–74
Breadth-First Search	63–67	75–79
Graph Foundations & Representations	68–73	80–85

Strongly Connected Components (SCC)

Kosaraju's 3-Pass Linear Time Algorithm in C++ (Lecture Notes & Analysis)

Executive Overview: A **Strongly Connected Component (SCC)** in a directed graph is a maximal subgraph where every vertex is mutually reachable from every other vertex within that component. While connectivity in undirected graphs is symmetric, directed graphs require specialized algorithms due to one-way edge orientations. **Kosaraju's Algorithm** resolves all SCCs in optimal linear time **O(V + E)** using a 3-step pipeline: finishing-time ordering via stack, graph transposition, and component extraction.

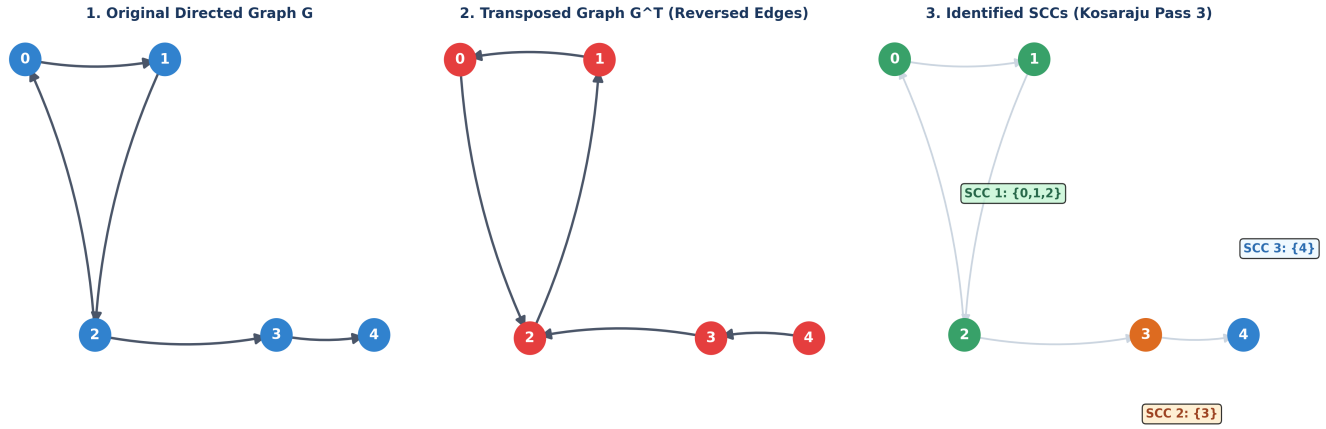


Figure 1: Original Directed Graph G, Transposed Graph G^T, and Extracted Strongly Connected Components.

1. The 3-Step Algorithmic Pipeline

Step 1: Finish-Time Stack Push: Run standard DFS on the original graph G. When a node finishes exploring all its adjacent neighbors, push it onto a LIFO stack upon recursive exit. The top of the stack will hold a vertex belonging to a source component in the component DAG.

Step 2: Edge Reversal (Graph Transposition): Reverse every directed edge u -> v in G to create v -> u in G^T. Transposition preserves internal reachability within each individual SCC while reversing cross-component transitions.

Step 3: Component Extraction: Pop vertices sequentially from the Step 1 stack. If a vertex is unvisited in G^T, initiate a DFS traversal starting from it. All vertices reachable in this pass form one complete Strongly Connected Component.

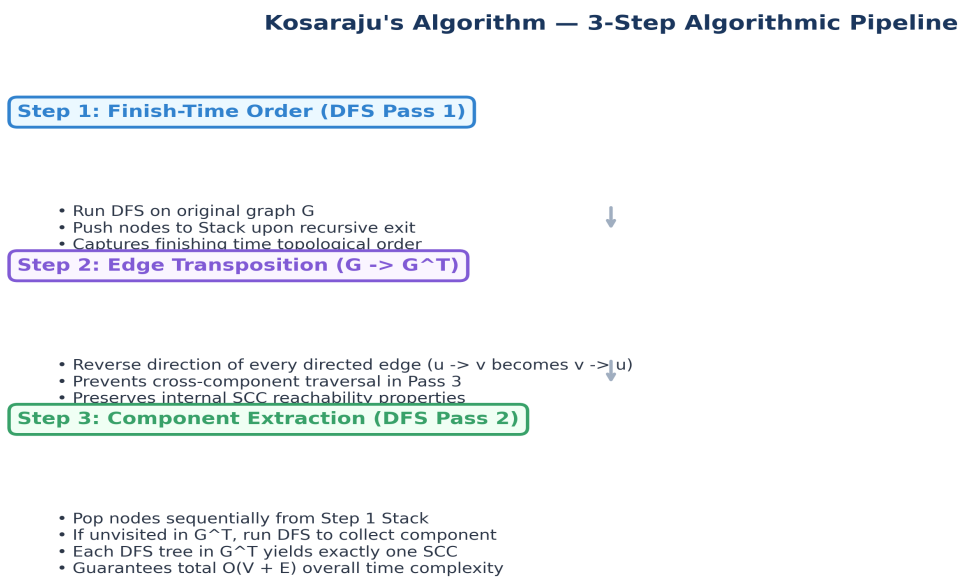


Figure 2: Architectural Pipeline and Execution Logic for Kosaraju's Algorithm.

2. Time & Space Complexity Analysis

Algorithmic Stage	Time Complexity	Space Complexity	Primary Purpose
Step 1: DFS Pass 1	O(V + E)	O(V)	Record finishing-time order on stack
Step 2: Transposition	O(V + E)	O(V + E)	Construct transposed graph G^T
Step 3: DFS Pass 2	O(V + E)	O(V)	Extract individual SCCs
Total Bounds	**O(V + E)**	**O(V + E)**	**Optimal Linear Bounds**

3. Production-Ready C++ Source Code

```
class Solution {
private:
    void dfsPass1(int node, vector adj[], vector& vis, stack& st) {
        vis[node] = true;
        for (int nbr : adj[node])
            if (!vis[nbr]) dfsPass1(nbr, adj, vis, st);
        st.push(node);
    }

    void dfsPass2(int node, vector adjT[], vector& vis, vector& scc) {
        vis[node] = true;
        scc.push_back(node);
        for (int nbr : adjT[node])
            if (!vis[nbr]) dfsPass2(nbr, adjT, vis, scc);
    }

public:
    vector> kosaraju(int V, vector adj[]) {
        stack st; vector vis(V, false);
        for (int i = 0; i < V; ++i)
            if (!vis[i]) dfsPass1(i, adj, vis, st);

        vector adjT[V];
        for (int u = 0; u < V; ++u)
            for (int v : adj[u]) adjT[v].push_back(u);

        fill(vis.begin(), vis.end(), false);
        vector> sccs;
        while (!st.empty()) {
            int u = st.top(); st.pop();
            if (!vis[u]) {
                vector scc;
                dfsPass2(u, adjT, vis, scc);
                sccs.push_back(scc);
            }
        }
        return sccs;
    }
};
```

Articulation Points in Graph using Tarjan's Algorithm

Critical Points, Dual Conditions (tin/low & Root Child Count), and C++ Implementation

1. Executive Summary & Problem Definition

An **Articulation Point** (or *Cut Vertex*) in an undirected graph is a vertex whose removal increases the number of connected components. Finding articulation points identifies single points of failure in server networks, communication backbones, and power grids.

While a naive brute-force vertex removal approach requires O(V * (V + E)) time, **Tarjan's Algorithm** identifies all critical points in a single DFS traversal in **O(V + E)** time and **O(V)** space.

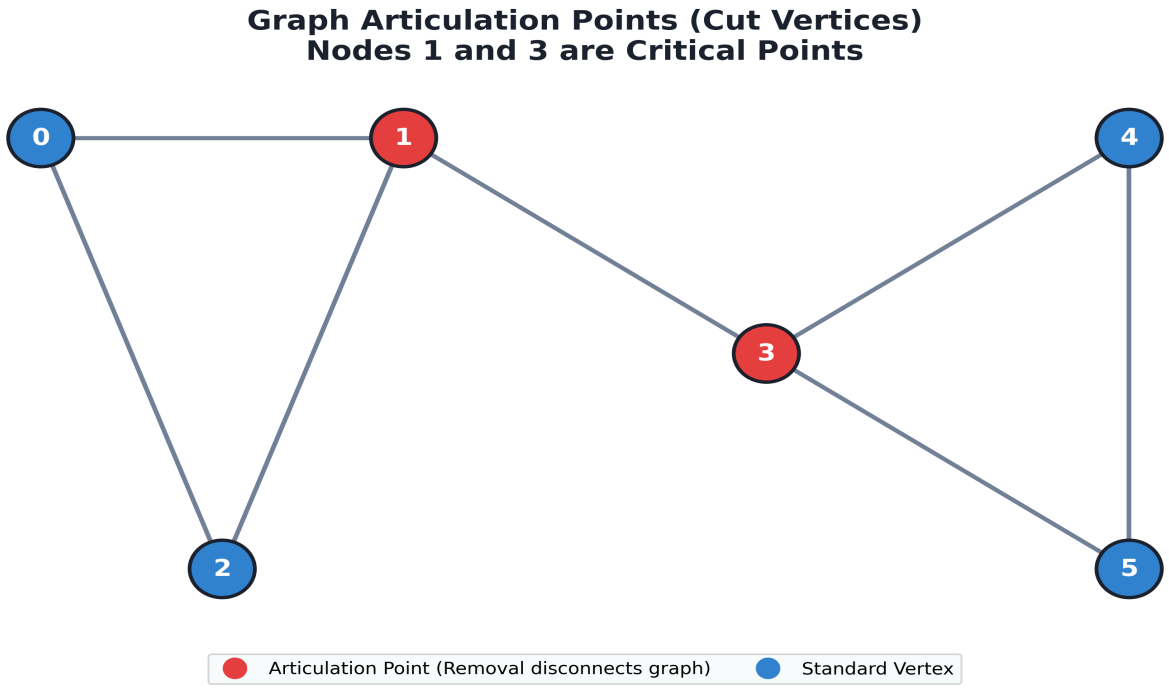


Figure 1: Graph with Articulation Points (Nodes 1 and 3 highlighted in red).

2. Tarjan's Dual Condition Logic

Tarjan's algorithm uses two auxiliary arrays: **tin[]** (insertion timestamp) and **low[]** (lowest discovery time reachable via subtree and back-edges). Two distinct rules govern articulation points:

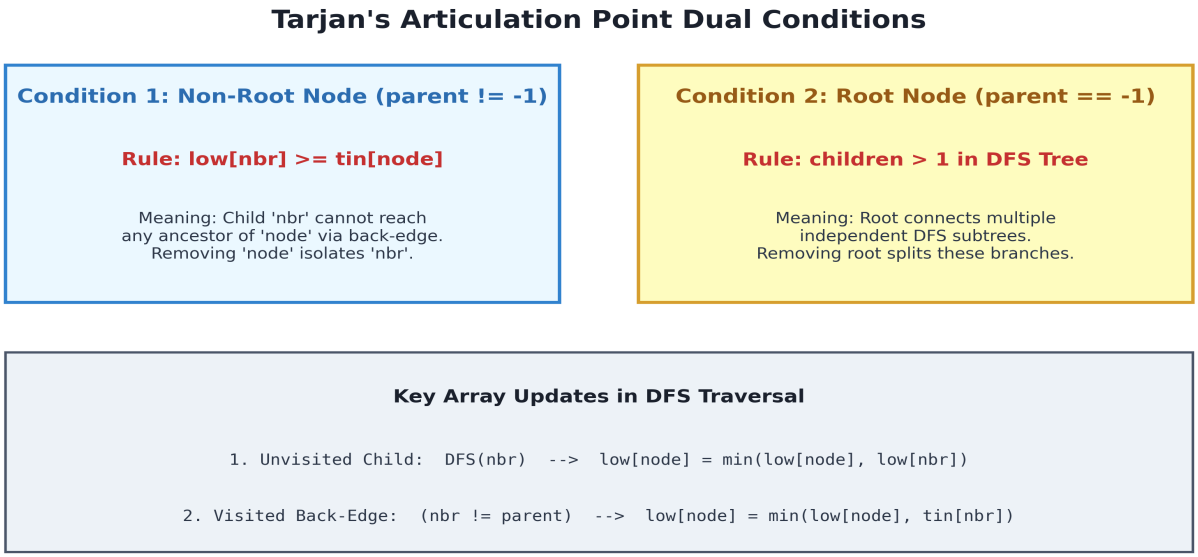


Figure 2: Non-Root vs Root Articulation Point Conditions.

3. Comparison: Articulation Points vs Bridges

Feature	Articulation Points (Cut Vertices)	Bridges (Critical Edges)
Target Entity	Vertex / Node	Edge / Connection
Condition Rule	low[nbr] >= tin[node]	low[nbr] > tin[node]
Root Special Case	childCount > 1 in DFS Tree	None (Standard rule applies)
Reporting Mechanism	Boolean array (prevents duplicates)	Direct edge pair collection
Time & Space Complexity	O(V + E) Time, O(V) Space	O(V + E) Time, O(V) Space

4. Production C++ Source Code

```
void dfs(int node, int parent, vector& tin, vector& low,
         vector& visited, vector& isArticulation) {
    visited[node] = true;
    tin[node] = low[node] = ++timer;
    int childCount = 0;

    for (int nbr : adj[node]) {
        if (nbr == parent) continue;

        if (visited[nbr]) {
            low[node] = min(low[node], tin[nbr]);
        } else {
```

```
            childCount++;
            dfs(nbr, node, tin, low, visited, isArticulation);
            low[node] = min(low[node], low[nbr]);

            if (low[nbr] >= tin[node] && parent != -1) {
                isArticulation[node] = true;
            }
        }
    }

    if (parent == -1 && childCount > 1) {
        isArticulation[node] = true;
    }
}
```

Tarjan's Algorithm: Bridges in Graph

Critical Connections in a Network (LeetCode 1192) | C++ Technical Guide

Executive Overview: A **Bridge** (or *Critical Connection*) in an undirected graph is an edge whose removal increases the number of connected components. Tarjan's Algorithm identifies all bridges in a single DFS pass with **O(V + E)** time complexity by tracking node insertion times (tin) and lowest reachable insertion times (low).

1. Fundamental Bridge Detection Logic

In a DFS traversal tree, as we explore from node *u* to neighbor *v*, we maintain two tracking arrays:
• **tin[node]:** The timestamp when *node* was first discovered in DFS.
• **low[node]:** The minimum discovery time tin reachable from *node* (including its subtree) via back-edges.

The Bridge Criterion: An edge **(u, v)** is a bridge if and only if:
low[v] > tin[u]
This strictly proves that neighbor *v* has no alternate back-edge path to reach *u* or any ancestor of *u*. Removing *(u, v)* completely severs *v* and its subtree from the rest of the graph.

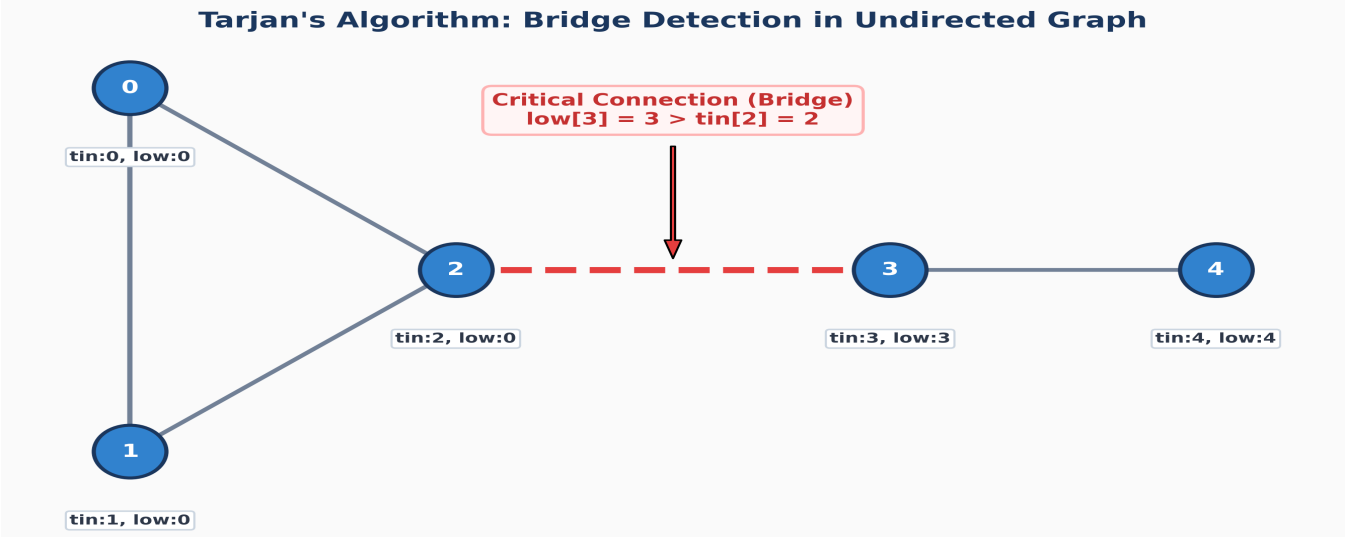


Figure 1: Graph visualization identifying critical bridge edge (2 -- 3) where low[3] > tin[2].

2. DFS State Trace & Timer Evolution

Node	Parent	tin	low	Bridge Check	Status
0	-1	0	0	Root Node	Visited

Node	Parent	tin	low	Bridge Check	Status
1	0	1	0 (via 2->0)	low[1] = 0 <= tin[0]	Cycle Member
2	1	2	0 (via 2->0)	low[2] = 0 <= tin[1]	Cycle Member
3	2	3	3	low[3] = 3 > tin[2] = 2	BRIDGE (2--3)
4	3	4	4	low[4] = 4 > tin[3] = 3	BRIDGE (3--4)

Tarjan's DFS Trace: tin vs low Array State Updates

Node	Parent	tin (Insertion Time)	low (Lowest Reachab	Bridge Condition Chec	Status
0	-1	0	0	Root Node	Visited
1	0	1	0 (via 2->0)	low[1] = 0 <= tin[0]	Cycle Member
2	1	2	0 (via 2->0)	low[2] = 0 <= tin[1]	Cycle Member
3	**2**	**3**	**3**	**low[3] = 3 > tin[2] =**	**BRIDGE (2--3)**
4	**3**	**4**	**4**	**low[4] = 4 > tin[3] =**	**BRIDGE (3--4)**

Figure 2: Comprehensive DFS state table tracing tin and low values across tree and back-edges.

3. Production C++ Implementation

```
class Solution {
private:
int timer = 0;
void dfs(int node, int parent, vector<int>& tin, vector<int>& low,
vector<bool>& visited, vector<vector<int>>& adj,
vector<vector<int>>& bridges) {
visited[node] = true;
tin[node] = low[node] = timer++;
for (int nbr : adj[node]) {
if (nbr == parent) continue;
if (visited[nbr]) {
low[node] = min(low[node], tin[nbr]); // Back-edge
} else {
dfs(nbr, node, tin, low, visited, adj, bridges);
low[node] = min(low[node], low[nbr]);
if (low[nbr] > tin[node]) {
bridges.push_back({node, nbr}); // Bridge found!
}
}
}
}
public:
vector<vector<int>> criticalConnections(int n, vector<vector<int>>& connections) {
vector<vector<int>> adj(n);
for (auto& c : connections) { adj[c[0]].push_back(c[1]); adj[c[1]].push_back(c[0]); }
vector<int> tin(n, -1), low(n, -1); vector<bool> visited(n, false);
vector<vector<int>> bridges;
for (int i = 0; i < n; ++i) if (!visited[i]) dfs(i, -1, tin, low, visited, adj, bridges);
return bridges;
}
};
```

4. Complexity Analysis

• **Time Complexity: O(V + E)** — Performs a single DFS pass visiting each vertex once and exploring each edge twice.

• **Space Complexity: O(V + E)** — Requires O(V + E) for adjacency list, O(V) for tin, low, and visited arrays, plus recursion stack space up to O(V).

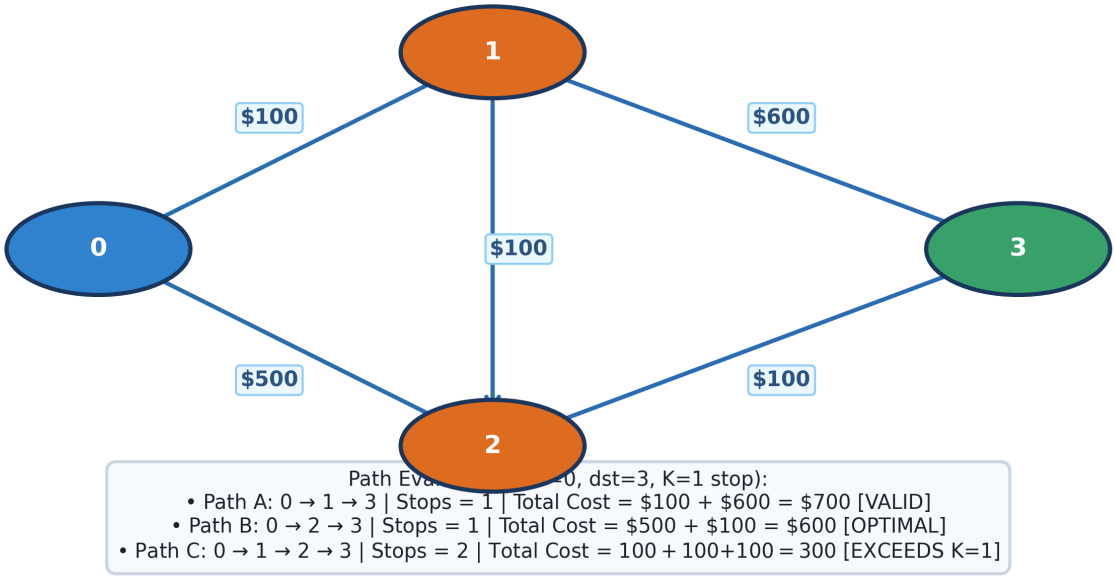
Cheapest Flights Within K Stops

LeetCode 787 — Graph Modeling, Constrained BFS & Distance Relaxation

1. Problem Statement & Graph Formulation

The **Cheapest Flights Within K Stops** problem requires finding the minimum total flight cost from a source city src to a destination city dst with at most k intermediate stops. A path with k stops contains at most k + 1 directed edges.

Cheapest Flights Within K Stops (LeetCode 787) — Graph & Constraints



2. Why Standard Dijkstra Fails Without Modification

Standard Dijkstra's algorithm uses a min-heap that pops nodes strictly by lowest accumulated cost. In constrained shortest path problems, a cheaper route with **more stops** can pop first, updating the distance array and causing the algorithm to prematurely discard a slightly more expensive route with **fewer stops** that could actually reach dst within k stops.

Feature / Strategy	Standard Dijkstra Algorithm	Level-Order BFS Queue Strategy
Priority / Queue Sorting	Min-Heap by total accumulated cost.	Level-by-level queue ordered by stop count ($0 \le \text{stops} \le k+1$).
Constraint Handling	Can prematurely prune paths with fewer stops.	Strictly prunes expansions when $\text{stops} > k$.

Time Complexity	$\mathcal{O}(E \log V)$	$\mathcal{O}(K \times E)$
Space Complexity	$\mathcal{O}(V + E)$	$\mathcal{O}(V + E)$

3. Execution Trace & State Evolution

Queue-Based BFS State Execution Trace (K = 1 Max Stops)

Level (Stops)	Popped (U, Cost)	Neighbors Explored	Updated Dist Vector	Queue State
Initial (0)	-	Push (src=0, cost=0)	[0, ∞, ∞, ∞]	[(0, 0, 0)]
Step 1 (Stops=0)	(0, $0)	1 (w=100), 2 (w=500)	[0, 100, 500, ∞]	[(1, 1, 100), (1, 2, 500)]
Step 2 (Stops=1)	(1, $100)	3 (100+600=700), 2 (100+100=2	[0, 100, 200, 700]	[(1, 2, $500), (2, 3, $700), (2, 2, $20(
Step 3 (Stops=1)	(2, $500)	3 (500+100=600 < 700)	[0, 100, 200, 600]	[(2, 3, $700), (2, 2, $200), (2, 3, $60(
Step 4 (Stops=2)	Stops > K=1	Pruned (Stops Exceeded)	Final Dist[3] = $600	Empty → Result: $600

4. Production C++ Source Code

```
class Solution { public: int findCheapestPrice(int n, vector>& flights, int src, int dst, int k) { vector>> adj(n); for (const auto& f : flights) { adj[f[0]].push_back({f[1], f[2]}); } vector dist(n, INT_MAX); dist[src] = 0; queue>> q; // {stops, {node, cost}} q.push({0, {src, 0}}); while (!q.empty()) { auto front = q.front(); q.pop(); int stops = front.first; int u = front.second.first; int cost = front.second.second; if (stops > k) continue; for (const auto& neighbor : adj[u]) { int v = neighbor.first, price = neighbor.second; if (cost + price < dist[v]) { dist[v] = cost + price; q.push({stops + 1, {v, dist[v]}}); } } } return dist[dst] == INT_MAX ? -1 : dist[dst]; } };
```

NUMBER OF PROVINCES (LEETCODE 547)

GRAPH CONNECTED COMPONENTS & MATRIX TRAVERSAL IN C++

1. Executive Overview & Problem Abstraction

The **Number of Provinces** problem (LeetCode 547) presents an $N \times N$ matrix isConnected representing direct city connections. In graph theory terms, a province represents a **maximal connected component** where all nodes in the component can reach each other directly or transitively.

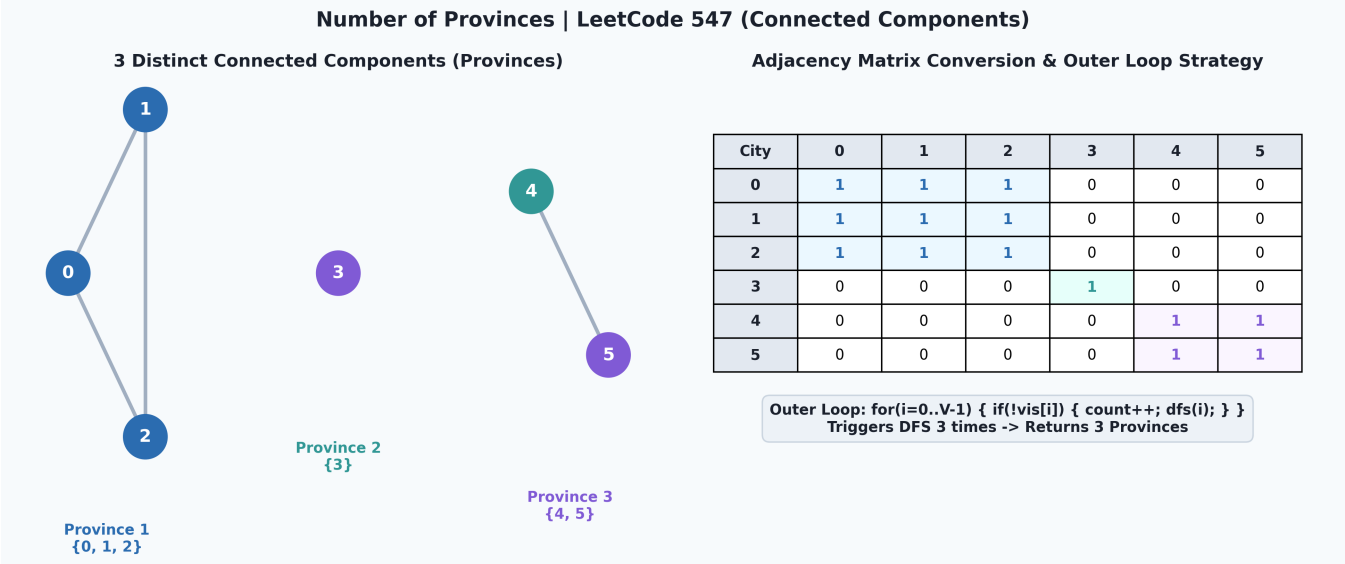


Figure 1: Graph Representation of Provinces & Adjacency Matrix Conversion

2. Algorithmic Paradigms Comparison

Strategy	Data Structure	Time Complexity	Space Complexity
Depth-First Search (DFS)	Recursion Call Stack	O(V^2)	O(V)
Breadth-First Search (BFS)	std::queue	O(V^2)	O(V)
Disjoint Set Union (DSU)	parent[] & rank[]	O(V^2 * alpha(V))	O(V)

3. Production C++ Implementation (DFS Approach)

```
#include <iostream>
#include <vector>

using namespace std;

class Solution {
private:
    void dfs(int node, const vector<vector<int>>& isConnected, vector<bool>& visited, int V) {
        visited[node] = true;
        for (int neighbor = 0; neighbor < V; ++neighbor) {
            if (isConnected[node][neighbor] == 1 && !visited[neighbor]) {
```

```
                dfs(neighbor, isConnected, visited, V);
            }
        }
    }

public:
    int findCircleNum(vector<vector<int>>& isConnected) {
        int V = isConnected.size();
        vector<bool> visited(V, false);
        int provinceCount = 0;

        for (int i = 0; i < V; ++i) {
            if (!visited[i]) {
                provinceCount++;
                dfs(i, isConnected, visited, V);
            }
        }
        return provinceCount;
    }
};
```

4. Technical Interview Takeaways

• **Outer Loop Trigger**: The main loop iterates over all vertices 0..V-1. Each unvisited node represents an unexplored province, triggering a new traversal.
• **Matrix Adjacency Scanning**: Because edges are specified in an $N \times N$ grid, neighbor iteration requires scanning all V columns rather than traversing a dynamic list.
• **Cycle Safety**: A visited boolean vector is mandatory to avoid infinitely revisiting connected components in undirected graphs.

Kruskal's Algorithm: Minimum Spanning Tree (MST)

Greedy Edge Sorting & Disjoint Set Union (DSU) in C++

Kruskal's Algorithm is a classic edge-centric **Greedy Algorithm** used to compute the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph. By sorting all edges in non-decreasing order of weight and systematically inserting them using the **Disjoint Set Union (DSU)** data structure with Path Compression and Union by Rank, Kruskal's algorithm avoids cycle formation and guarantees a minimal edge weight sum in **O(E log E)** time.

1. Algorithmic Mechanics & Disjoint Set Integration

Unlike Prim's Algorithm (which grows a single spanning tree from a starting vertex), Kruskal's Algorithm operates globally over all graph edges. The algorithm initializes every vertex into its own independent disjoint set. Edges are sorted by weight, and for each candidate edge (u, v, w), the algorithm executes find(u) == find(v). If the endpoints belong to distinct sets, the edge is accepted into the MST and the sets are merged using unionByRank(u, v). If the endpoints share a representative root, the edge is rejected as a back-edge that would form an invalid cycle.

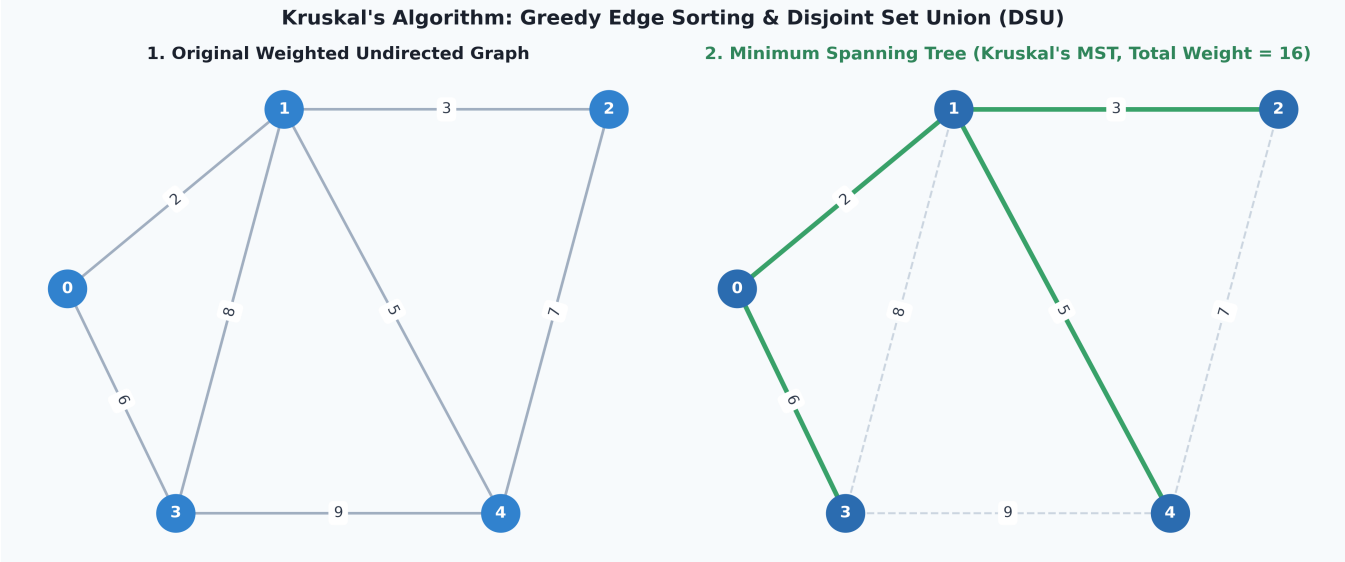


Figure 1: Kruskal's Minimum Spanning Tree Construction on a 5-Vertex Graph

2. Step-by-Step Execution Trace Table

Step	Edge (u, v)	Weight	find(u) == find(v)?	Decision / Action	MST Total	DSU Sets
1	(0, 1)	2	find(0)=0 != find(1)=1	ACCEPT (Union 0, 1)	2	{0,1}, {2}, {3}, {4}

Step	Edge (u, v)	Weight	find(u) == find(v)?	Decision / Action	MST Total	DSU Sets
2	(1, 2)	3	find(1)=0 != find(2)=2	ACCEPT (Union 1, 2)	5	{0,1,2}, {3}, {4}
3	(1, 4)	5	find(1)=0 != find(4)=4	ACCEPT (Union 1, 4)	10	{0,1,2,4}, {3}
4	(0, 3)	6	find(0)=0 != find(3)=3	ACCEPT (Union 0, 3)	16 (Complete)	{0,1,2,3,4}
5	(2, 4)	7	find(2)=0 == find(4)=0	REJECT (Cycle)	16	{0,1,2,3,4}
6	(1, 3)	8	find(1)=0 == find(3)=0	REJECT (Cycle)	16	{0,1,2,3,4}
7	(3, 4)	9	find(3)=0 == find(4)=0	REJECT (Cycle)	16	{0,1,2,3,4}

Kruskal's Algorithm: Sorted Edge Processing & DSU State Trace

Step	Edge (u, v)	Weight	find(u) == find(v)?	Action Taken	Total MST Weight	DSU Components
1	(0, 1)	2	find(0)=0 != find(1)=1	ACCEPT (Union 0, 1)	2	{0,1}, {2}, {3}, {4}
2	(1, 2)	3	find(1)=0 != find(2)=2	ACCEPT (Union 1, 2)	5	{0,1,2}, {3}, {4}
3	(1, 4)	5	find(1)=0 != find(4)=4	ACCEPT (Union 1, 4)	10	{0,1,2,4}, {3}
4	(0, 3)	6	find(0)=0 != find(3)=3	ACCEPT (Union 0, 3)	16 (MST Complete)	{0,1,2,3,4}
5	(2, 4)	7	find(2)=0 == find(4)=0	REJECT (Forms Cycle)	16	{0,1,2,3,4}
6	(1, 3)	8	find(1)=0 == find(3)=0	REJECT (Forms Cycle)	16	{0,1,2,3,4}
7	(3, 4)	9	find(3)=0 == find(4)=0	REJECT (Forms Cycle)	16	{0,1,2,3,4}

Figure 2: Edge Sorting Order & Disjoint Set State Transition Analysis

3. Production C++ Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Edge {
    int u, v, weight;
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

class DisjointSet {
private:
    vector<int> parent, rank;
public:
    DisjointSet(int n) {
        parent.resize(n);
        rank.assign(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int node) {
        if (node == parent[node]) return node;
        return parent[node] = find(parent[node]);
    }
    bool unionByRank(int u, int v) {
        int rootU = find(u), rootV = find(v);
        if (rootU == rootV) return false;
        if (rank[rootU] < rank[rootV]) parent[rootU] = rootV;
        else if (rank[rootU] > rank[rootV]) parent[rootV] = rootU;
        else { parent[rootV] = rootU; rank[rootU]++; }
        return true;
    }
};

class KruskalMST {
public:
    static pair<int, vector<Edge>> computeMST(int V, vector<Edge>& edges) {
        sort(edges.begin(), edges.end());
        DisjointSet dsu(V);
        int mstWeight = 0;
        vector<Edge> mstEdges;
        for (const auto& edge : edges) {
            if (dsu.unionByRank(edge.u, edge.v)) {
                mstWeight += edge.weight;
                mstEdges.push_back(edge);
                if (mstEdges.size() == V - 1) break;
            }
        }
        return {mstWeight, mstEdges};
    }
};
```

4. Kruskal's vs. Prim's Algorithm Comparison

Dimension	Kruskal's Algorithm	Prim's Algorithm
Strategy Paradigm	Edge-centric Greedy Selection	Vertex-centric Greedy Expansion
Data Structures	Sorted Edge Vector + DSU	Adjacency List + Min-Heap Priority Queue

Dimension	Kruskal's Algorithm	Prim's Algorithm
Optimal Graph Density	Sparse Graphs ($E \approx V$)	Dense Graphs ($E \approx V^2$)
Component Growth	Forest of trees merging together	Single tree growing outward from root
Time Complexity	O(E log E) or O(E log V)	O((V + E) log V)

Disjoint Set Union (DSU) in C++

Rank, Size, Path Compression & Inverse Ackermann Complexity

> **Key Takeaway:** Disjoint Set Union (DSU) maintains dynamic non-overlapping set partitions. By combining **Path Compression** with **Union by Rank/Size**, both find() and union() operations achieve near-constant amortized time complexity **O(α(N))**, where α(N) ≤ 4 for all practical inputs.

1. Core Mechanics & Naive Disjoint Set Pitfalls

Disjoint Set Union manages dynamic connectivity through two principal operations: find(u) (locating the ultimate root representative of element *u*) and union(u, v) (merging the sets containing *u* and *v*). In an unoptimized naive implementation, parent pointers can form a degenerate linear chain of length *N*, causing find() queries to degrade to linear time **O(N)**.

2. Path Compression & Union Strategy

To prevent tree height expansion, DSU applies two key optimization techniques:
1. **Path Compression:** Re-links every node along the recursion path directly to the ultimate root during find().
2. **Union by Rank / Size:** Attaches the smaller height (or size) tree under the root of the larger tree, ensuring balanced growth.



Figure 1: Tree flattening effect before and after Path Compression.

Disjoint Set Union (DSU) Array State & Operations Trace

Node Index [i	0	1	2	3	4	5	6	7
parent[i]	0	1	2	3	4	5	6	7
rank[i]	0	0	0	0	0	0	0	0
size[i]	1	1	1	1	1	1	1	1

Core Operations:
1. findParent(u): Uses Path Compression (parent[u] = findParent(parent[u]))
2. unionByRank(u, v) / unionBySize(u, v): Connects smaller tree under larger tree root

Figure 2: Array representations (`parent`, `rank`, `size`) for DSU operations.

3. Optimization & Complexity Comparison

Optimization Strategy	Time Complexity per Operation	Space Complexity
Naive Union-Find	O(N) Worst-Case	O(N)
Path Compression Only	O(log N) Amortized	O(N)
Union by Rank / Size Only	O(log N) Worst-Case	O(N)
Path Compression + Rank/Size	O(α(N)) Amortized (≈ O(1))	O(N)

4. Production-Ready Object-Oriented C++ Code

```
#include <iostream>
#include <vector>
#include <numeric>

using namespace std;

class DisjointSet {
private:
    vector<int> parent, rank, size;
public:
    DisjointSet(int n) {
        parent.resize(n + 1);
        iota(parent.begin(), parent.end(), 0);
        rank.assign(n + 1, 0);
        size.assign(n + 1, 1);
    }

    int findUltimateParent(int node) {
        if (node == parent[node]) return node;
```

```
        return parent[node] = findUltimateParent(parent[node]);
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUltimateParent(u);
        int ulp_v = findUltimateParent(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        } else {
            parent[ulp_v] = ulp_u;
            size[ulp_u] += size[ulp_v];
        }
    }
};
```

Prim's Algorithm — Minimum Spanning Tree (MST)

Data Structures & Algorithms | C++ Implementation Guide

> **Key Takeaway:** Prim's Algorithm is a greedy graph algorithm that constructs a **Minimum Spanning Tree (MST)** for a connected, weighted, undirected graph. Starting from an arbitrary node, it iteratively adds the minimum-weight edge connecting a visited node to an unvisited node using a **Min-Heap Priority Queue**.

1. Spanning Tree Fundamentals & Cut Property

A **spanning tree** of a graph *G = (V, E)* is an acyclic connected subgraph containing all *V* vertices and exactly *V - 1* edges. The **Minimum Spanning Tree (MST)** minimizes the cumulative edge weight across all possible spanning trees.

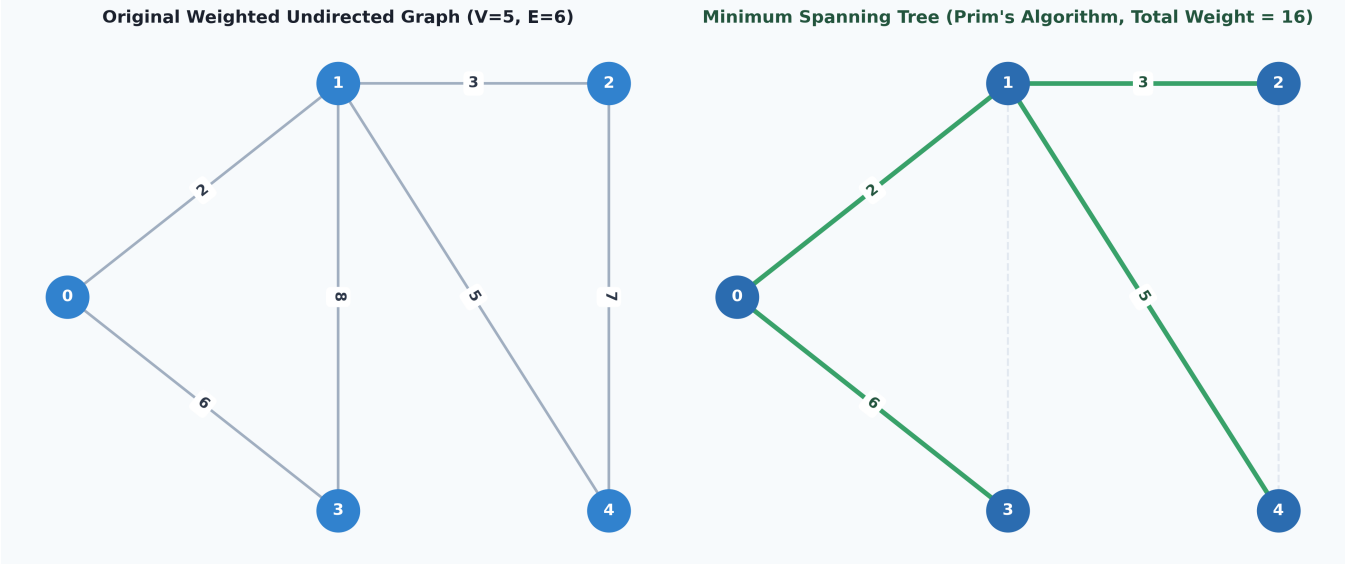


Figure 1: Original weighted graph vs. Prim's Minimum Spanning Tree (Total Weight = 16)

2. Min-Heap Execution Step-by-Step Trace

Prim's algorithm maintains a priority queue of (weight, node) pairs to extract the light edge at each cut:

Prim's Algorithm Priority Queue (Min-Heap) Step-by-Step Trace

Min-Heap Stores Pairs: (Edge Weight, Vertex Node)

Step 1	Pick start vertex 0. Mark vis[0]=true. Push edges (2,1), (6,3) into Min-Heap.	**PQ: [(2,1), (6,3)]**	**MST Weight: 0**
Step 2	Pop (2,1). Visited[1]=false -> Include Node 1 (Weight +2). Mark vis[1]=true. Push unvisited neighbors: (3,2), (5,4), (8,3).	**PQ: [(3,2), (5,4), (6,3), (8,3)]**	**MST Weight: 2**
Step 3	Pop (3,2). Visited[2]=false -> Include Node 2 (Weight +3). Mark vis[2]=true. Push unvisited neighbors: (7,4).	**PQ: [(5,4), (6,3), (7,4), (8,3)]**	**MST Weight: 5**
Step 4	Pop (5,4). Visited[4]=false -> Include Node 4 (Weight +5). Mark vis[4]=true.	**PQ: [(6,3), (7,4), (8,3)]**	**MST Weight: 10**
Step 5	Pop (6,3). Visited[3]=false -> Include Node 3 (Weight +6). Mark vis[3]=true.	**PQ: [(7,4), (8,3)]**	**MST Weight: 16**
Step 6	Pop remaining items (7,4), (8,3). Since vis[4] and vis[3] are true, skip.	**PQ: Empty**	**Final MST Weight: 16**

Figure 2: Step-by-step min-heap priority queue trace and weight accumulation

3. Production-Ready C++ Source Code

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

class Graph {
private:
    int V;
    vector<pair<int, int>>* adj; // Adjacency list: {neighbor, weight}

public:
    Graph(int vertices) {
        this->V = vertices;
        adj = new vector<pair<int, int>>[V];
    }

    ~Graph() { delete[] adj; }

    void addEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
        adj[v].push_back({u, w}); // Undirected
    }

    int primMST() {
        // Min-heap priority queue: (weight, node)
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
        vector<bool> visited(V, false);
        int mstWeight = 0;

        pq.push({0, 0}); // Start at node 0 with weight 0

        while (!pq.empty()) {
            auto [weight, u] = pq.top();
            pq.pop();

            if (visited[u]) continue;

            visited[u] = true;
```

```
            mstWeight += weight;

            for (auto& neighbor : adj[u]) {
                int v = neighbor.first;
                int w = neighbor.second;
                if (!visited[v]) {
                    pq.push({w, v});
                }
            }
        }
        return mstWeight;
    }
};
```

4. Comparative Analysis: Prim's vs. Kruskal's vs. Dijkstra's

Algorithm	Primary Target	Greedy Strategy	Time Complexity
Prim's Algorithm	Minimum Spanning Tree (MST)	Min-heap vertex cut expansion	O((V + E) log V)
Kruskal's Algorithm	Minimum Spanning Tree (MST)	Global edge sorting + Disjoint Set Union	O(E log E)
Dijkstra's Algorithm	Single Source Shortest Path	Min-heap path distance relaxation	O((V + E) log V)

Bellman-Ford Algorithm: SSSP & Dynamic Programming

Single-Source Shortest Path, Negative Edge Weights & Negative Cycle Detection in C++

1. Core Principles & Problem Formulation

The **Bellman-Ford Algorithm** computes the single-source shortest path (SSSP) from a starting vertex to all other vertices in a weighted directed or undirected graph. Unlike Dijkstra's algorithm, which requires all edge weights to be non-negative, Bellman-Ford correctly handles negative edge weights and provides a mechanism to detect negative weight cycles.

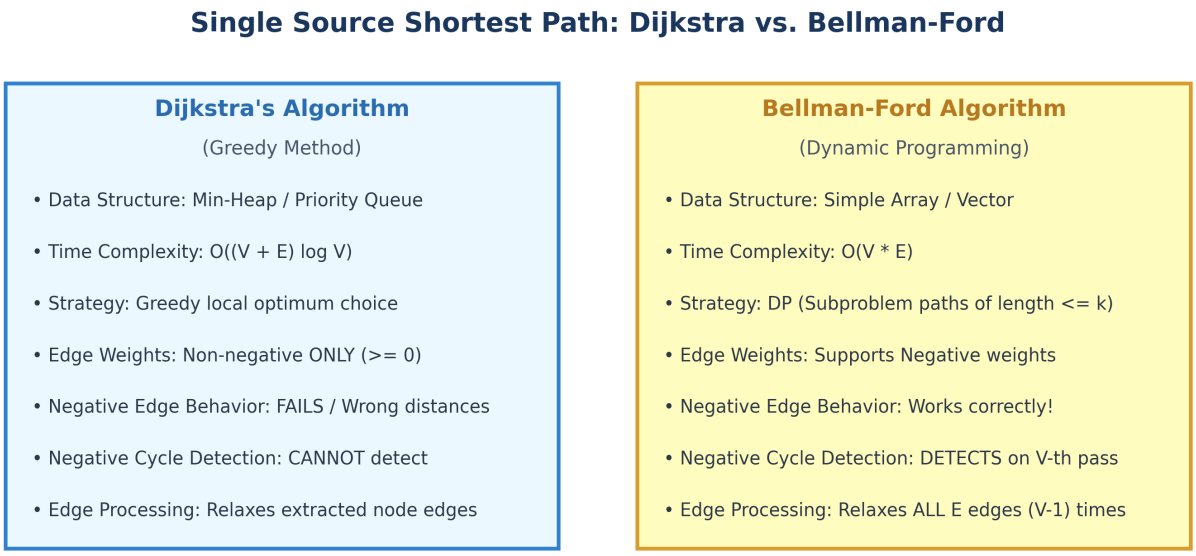


Figure 1: Comparative Analysis of Dijkstra vs. Bellman-Ford Algorithms.

2. Dynamic Programming Paradigm & Edge Relaxation

Bellman-Ford operates on a **Dynamic Programming** paradigm. In a simple graph with V vertices, any shortest path contains at most (V - 1) edges. Thus, relaxing all edges (V - 1) times guarantees that shortest path distances for all reachable vertices are computed.

Relaxation Rule: For every edge e = (u, v, w), if dist[u] != ∞ and dist[u] + w < dist[v], then update dist[v] = dist[u] + w.

3. Algorithm Trace & Negative Cycle Detection

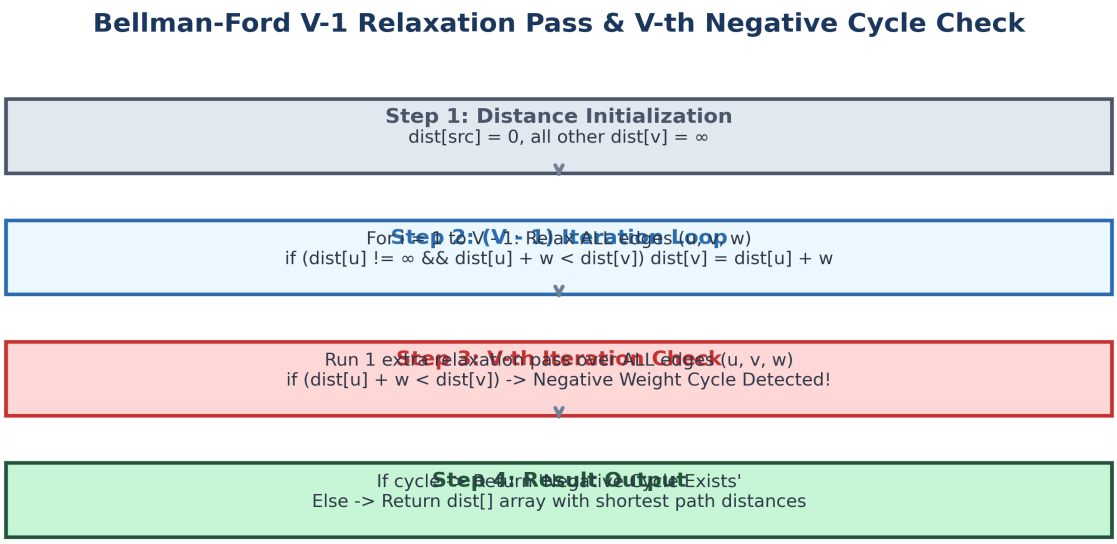


Figure 2: Bellman-Ford Iterative Relaxation Flow and V-th Iteration Negative Cycle Check.

4. Shortest Path Algorithms Comparison Matrix

Feature	Dijkstra's Algorithm	Bellman-Ford Algorithm	Floyd-Warshall Algorithm
Problem Type	Single-Source Shortest Path	Single-Source Shortest Path	All-Pairs Shortest Path
Strategy	Greedy	Dynamic Programming	Dynamic Programming
Time Complexity	O((V + E) log V)	O(V * E)	O(V^3)
Space Complexity	O(V)	O(V)	O(V^2)
Negative Edges	Not Supported (Fails)	Supported (Works)	Supported (Works)
Negative Cycles	Cannot Detect	Detects on V-th pass	Detects via dist[i][i] < 0

5. Object-Oriented C++ Implementation

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

struct Edge { int u; int v; int w; };

class BellmanFord {
private:
    int V;
    vector<Edge> edges;
```

```
public:
    BellmanFord(int vertices) : V(vertices) {}
    void addEdge(int u, int v, int w) { edges.push_back({u, v, w}); }

    void solve(int src) {
        vector<int> dist(V, INT_MAX);
        dist[src] = 0;

        // Step 1: Relax all edges (V - 1) times
        for (int i = 1; i <= V - 1; ++i) {
            for (const auto& edge : edges) {
                if (dist[edge.u] != INT_MAX && dist[edge.u] + edge.w < dist[edge.v]) {
                    dist[edge.v] = dist[edge.u] + edge.w;
                }
            }
        }

        // Step 2: V-th iteration check for negative cycles
        for (const auto& edge : edges) {
            if (dist[edge.u] != INT_MAX && dist[edge.u] + edge.w < dist[edge.v]) {
                cout << "[ALERT] Negative Weight Cycle Detected!\n";
                return;
            }
        }

        // Print shortest path distances
        for (int i = 0; i < V; ++i) cout << "Vertex " << i << ": " << dist[i] << "\n";
    }
};
```

Dijkstra's Shortest Path Algorithm

Single-Source Shortest Path in Weighted Graphs (C++ DSA)

1. Algorithmic Overview & Mathematical Foundations

Dijkstra's Algorithm is a foundational single-source shortest path algorithm operating on weighted graphs with non-negative edge weights. Given a graph *G = (V, E)* and a source node *S*, Dijkstra greedily computes the shortest distance from *S* to all other vertices.

The core computational step is **Edge Relaxation**. For a vertex *u* with distance *dist[u]*, relaxation evaluates every outgoing neighbor *v* with edge weight *w(u, v)*:

if (dist[u] + w(u, v) < dist[v]) { dist[v] = dist[u] + w(u, v); pq.push({dist[v], v}); }

> **Critical Constraint:** Dijkstra's algorithm requires all edge weights to be **non-negative** (*w(u, v) ≥ 0*). If negative edge weights exist, Bellman-Ford algorithm must be used instead to handle potential negative cycles.

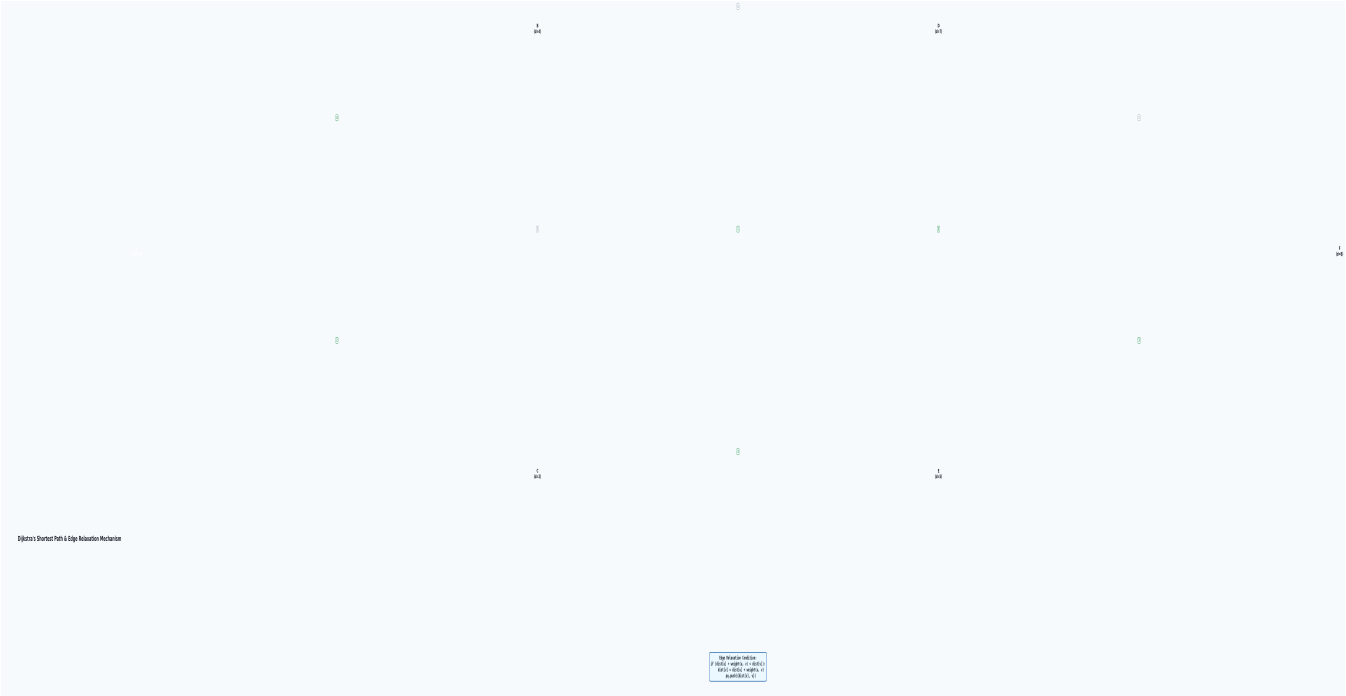


Figure 1: Dijkstra's Graph Structure & Edge Relaxation Mechanism

2. Min-Heap Priority Queue & Step-by-Step State Trace

To efficiently extract the unvisited vertex with the minimum tentative distance, C++ STL **std::priority_queue** is configured as a min-heap using *std::greater<pair<int, int>>* storing *(distance, node)* pairs.

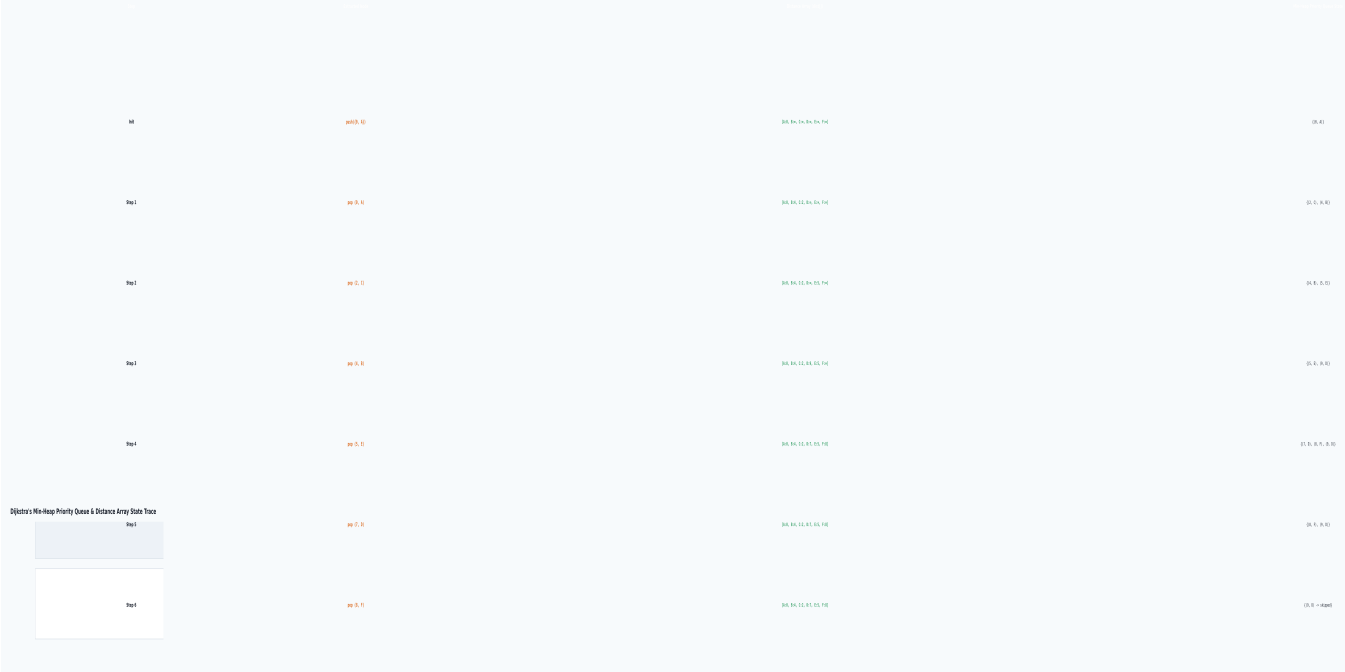


Figure 2: Min-Heap Priority Queue State Trace & Distance Array Updates

3. Production-Ready C++ Implementation

Below is a complete, object-oriented C++ implementation of Dijkstra's algorithm using an Adjacency List and Min-Heap Priority Queue:

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>

using namespace std;

class WeightedGraph {
private:
    int V;
    vector<pair<int, int>>* adj; // pair<neighbor, weight>

public:
    WeightedGraph(int vertices) {
        this->V = vertices;
        adj = new vector<pair<int, int>>[V];
    }
    ~WeightedGraph() { delete[] adj; }

    void addEdge(int u, int v, int w, bool isDirected = true) {
        adj[u].push_back({v, w});
        if (!isDirected) adj[v].push_back({u, w});
    }

    vector<int> dijkstra(int src) {
        vector<int> dist(V, INT_MAX);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

        dist[src] = 0;
        pq.push({0, src});

        while (!pq.empty()) {
            auto [current_dist, u] = pq.top();
            pq.pop();

            if (current_dist > dist[u]) continue; // Skip outdated entry

            for (auto& edge : adj[u]) {
                int v = edge.first;
                int weight = edge.second;

                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    pq.push({dist[v], v});
                }
            }
        }
        return dist;
    }
};
```

4. Complexity Analysis & Technical Takeaways

Metric / Dimension	Complexity	Technical Explanation
Time Complexity	O((V + E) log V)	V min-heap pop operations + E edge relaxations, each taking O(log V) time.

Space Complexity	O(V + E)	Adjacency list storage O(V + E) plus distance array O(V) and priority queue O(V).

Topological Sorting using Kahn's Algorithm in C++

Level-Order Breadth-First Search Driven by Indegree Elimination | LeetCode & DSA Guide

1. Core Algorithmic Concept: Indegree Elimination

Topological sorting represents a linear ordering of vertices in a **Directed Acyclic Graph (DAG)** such that for every directed edge $u \rightarrow v$, vertex u appears before vertex v. While DFS-based approaches use backtracking call stacks, **Kahn's Algorithm** models topological sorting through an iterative **Breadth-First Search (BFS)** strategy powered by **in-degrees** (number of incoming directed edges).

> **Key Principle:** A vertex with an **indegree of 0** has no incoming dependencies or prerequisites. Kahn's Algorithm initializes a FIFO queue with all 0-indegree nodes, popping them iteratively while decrementing neighbor indegrees. If the output array size equals total vertices V, topological ordering succeeds; otherwise, a directed cycle exists.

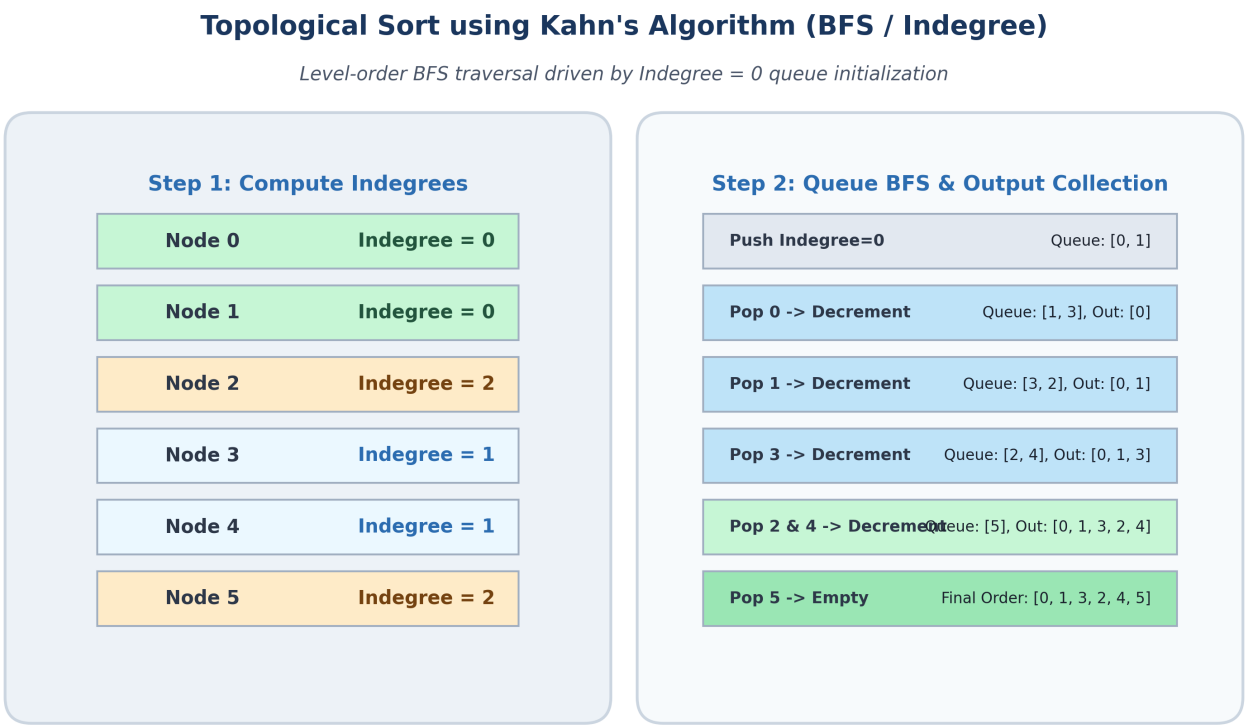


Figure 1: Kahn's Algorithm Indegree Calculation and Queue State Trace.

2. Dry-Run Execution Trace

Consider a DAG with $V = 6$ nodes and directed edges: $5 \to 0, 5 \to 2, 4 \to 0, 4 \to 1, 2 \to 3, 3 \to 1$.

Step	Action / Pop	Indegree Updates	Queue State	Output Sequence
Initial	—	0: 2, 1: 2, 2: 1, 3: 1, 4: 0, 5: 0	[4, 5]	[]
1	Pop 4	Neighbor 0: 21, Neighbor 1: 21	[5]	[4]
2	Pop 5	Neighbor 0: 10 (push 0), 2: 10 (push 2)	[0, 2]	[4, 5]
3	Pop 0	No outgoing neighbors	[2]	[4, 5, 0]
4	Pop 2	Neighbor 3: 10 (push 3)	[3]	[4, 5, 0, 2]
5	Pop 3	Neighbor 1: 10 (push 1)	[1]	[4, 5, 0, 2, 3]
6	Pop 1	No outgoing neighbors	[]	[4, 5, 0, 2, 3, 1]

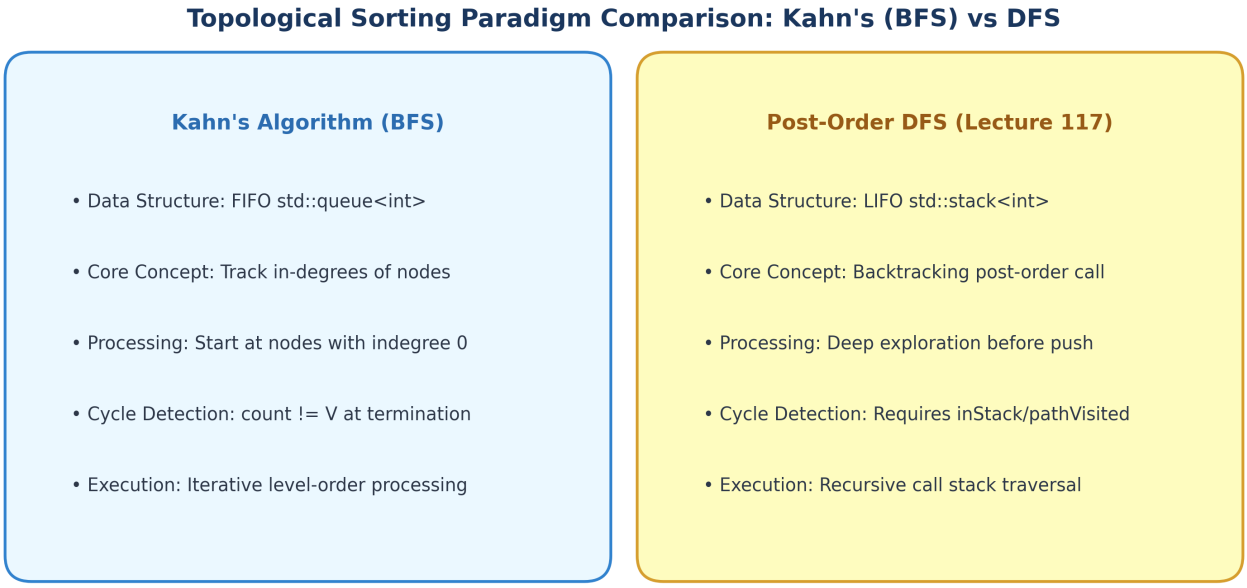


Figure 2: Architectural Comparison between Kahn's Algorithm (BFS) and Post-Order DFS.

3. Production-Ready C++ Source Code

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

public:
    Graph(int vertices) : V(vertices), adj(vertices) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v); // Directed edge u -> v
    }

    vector<int> topologicalSortKahn() {
        vector<int> indegree(V, 0);

        // Calculate indegrees
        for (int i = 0; i < V; ++i) {
            for (int neighbor : adj[i]) {
                indegree[neighbor]++;
            }
        }

        // Push 0-indegree nodes
        queue<int> q;
        for (int i = 0; i < V; ++i) {
            if (indegree[i] == 0) q.push(i);
        }

        vector<int> topoOrder;
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            topoOrder.push_back(curr);

            for (int neighbor : adj[curr]) {
                indegree[neighbor]--;
                if (indegree[neighbor] == 0) q.push(neighbor);
            }
        }

        if (topoOrder.size() != V) return {}; // Cycle detected
        return topoOrder;
    }
};
```

4. Time & Space Complexity

Time Complexity: $\mathcal{O}(V + E)$, where V is the number of vertices and E is the number of directed edges. Every node and edge is processed exactly once.
Space Complexity: $\mathcal{O}(V)$, required for the indegree array, output vector, and FIFO queue.

FLOOD FILL ALGORITHM IN C++

2D Grid Graph Traversal, Connected Components & LeetCode 733 Architecture

1. Problem Formulation & Grid Graph Abstraction

The **Flood Fill Algorithm** (LeetCode 733) models a 2D matrix as an implicit graph where each pixel cell is a vertex and 4-directionally adjacent cells sharing the initial starting color form undirected edges. It serves as the core algorithm powering paint bucket tools in graphics applications.

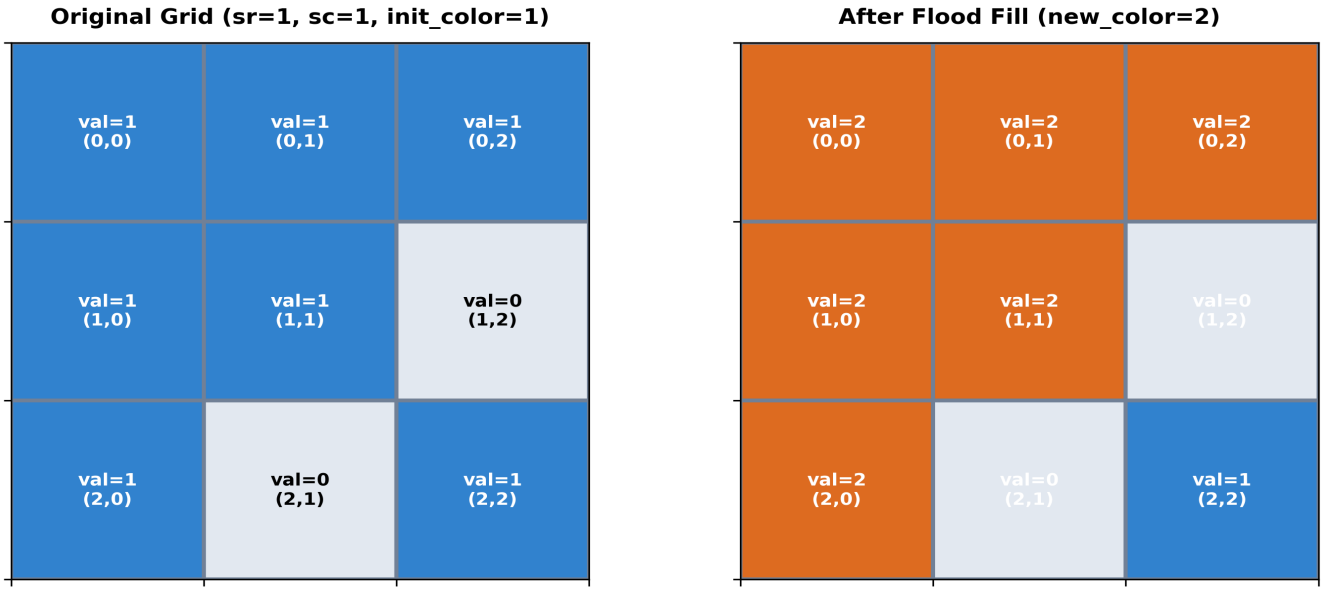


Figure 1: 2D Grid Transformation during Flood Fill (sr=1, sc=1, newColor=2).

2. Critical Edge Case: Infinite Recursion Guard

A critical pitfall in Flood Fill occurs when the starting pixel's original color equals the target color (image[sr][sc] == color). Without an explicit guard check, the algorithm repeatedly re-processes recolored nodes, resulting in infinite recursion or queue overflow.

> **CRITICAL GUARD CONDITION:**
> if (image[sr][sc] == color) return image;
> Checking this guard condition prior to starting traversal guarantees algorithm termination and prevents stack overflow.

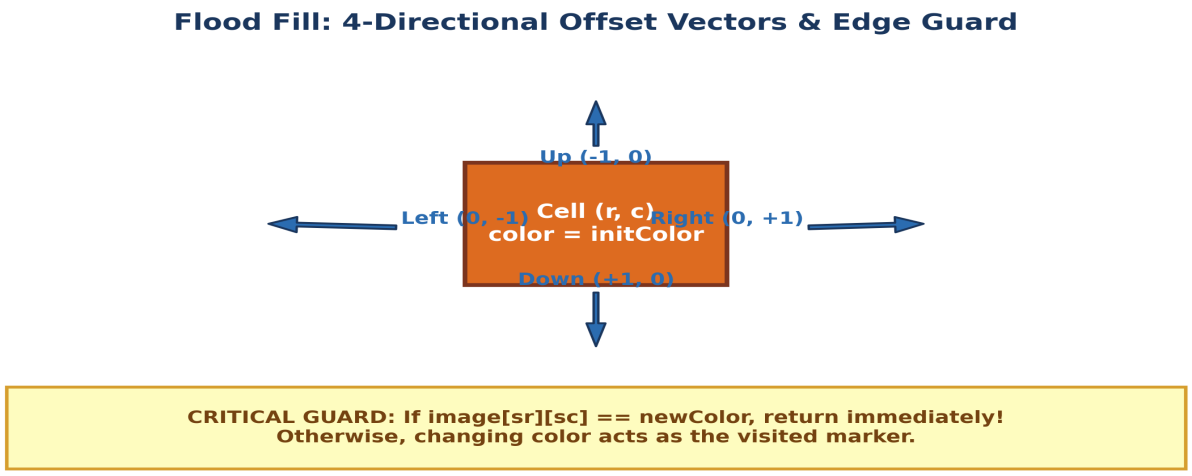


Figure 2: 4-Directional Offset Vectors and Recursion Guard Logic.

3. Complete C++ DFS Code Implementation

```
// Production C++ DFS Implementation for LeetCode 733
#include <iostream>
#include <vector>
using namespace std;

class Solution {
private:
    const int dr[4] = {-1, 0, 1, 0};
    const int dc[4] = {0, 1, 0, -1};

    void dfs(vector<vector<int>>& image, int r, int c, int initColor, int newColor, int m, int n) {
        image[r][c] = newColor;
        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n && image[nr][nc] == initColor) {
                dfs(image, nr, nc, initColor, newColor, m, n);
            }
        }
    }
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int color) {
        int initColor = image[sr][sc];
        if (initColor == color) return image; // Guard
        dfs(image, sr, sc, initColor, color, image.size(), image[0].size());
        return image;
    }
};
```

4. Algorithmic Complexity Proofs

Metric	Complexity	Technical Justification
Time Complexity	O(M x N)	Every grid cell is visited and recolored at most once.
Space Complexity	O(M x N)	Auxiliary stack space (DFS) or queue space (BFS) worst case equals grid size.

Course Schedule Problem (LeetCode 207 & 210)

Graph Abstraction, Kahn's Topological Sort, BFS Indegree & C++ Implementation

The Course Schedule problem (LeetCode 207: Course Schedule I and LeetCode 210: Course Schedule II) represents one of the most practical and frequently asked graph interview problems. It models course prerequisite dependencies as a Directed Graph, requiring developers to detect cycles or compute a valid linear course completion order using Topological Sorting.

> **Core Rule:** A valid course schedule exists if and only if the directed prerequisite graph is a **Directed Acyclic Graph (DAG)**. If a cycle exists, a deadlock occurs, making completion impossible.

1. Graph Problem Abstraction

Given N courses (0 to N-1) and prerequisite pairs [a, b] (where course b must be taken before course a), we represent the problem as follows:

Graph Entity	Course Schedule Mapping	Real-World Significance
Vertices (V)	Individual Courses (0 to N-1)	Tasks, Courses, Modules
Directed Edges (E)	Prerequisite Edge: b a	Dependency requirement
Indegree	Count of incoming edges to course a	# of unfinished prerequisites
Topological Order	Valid sequence respecting all edges	Executable course sequence

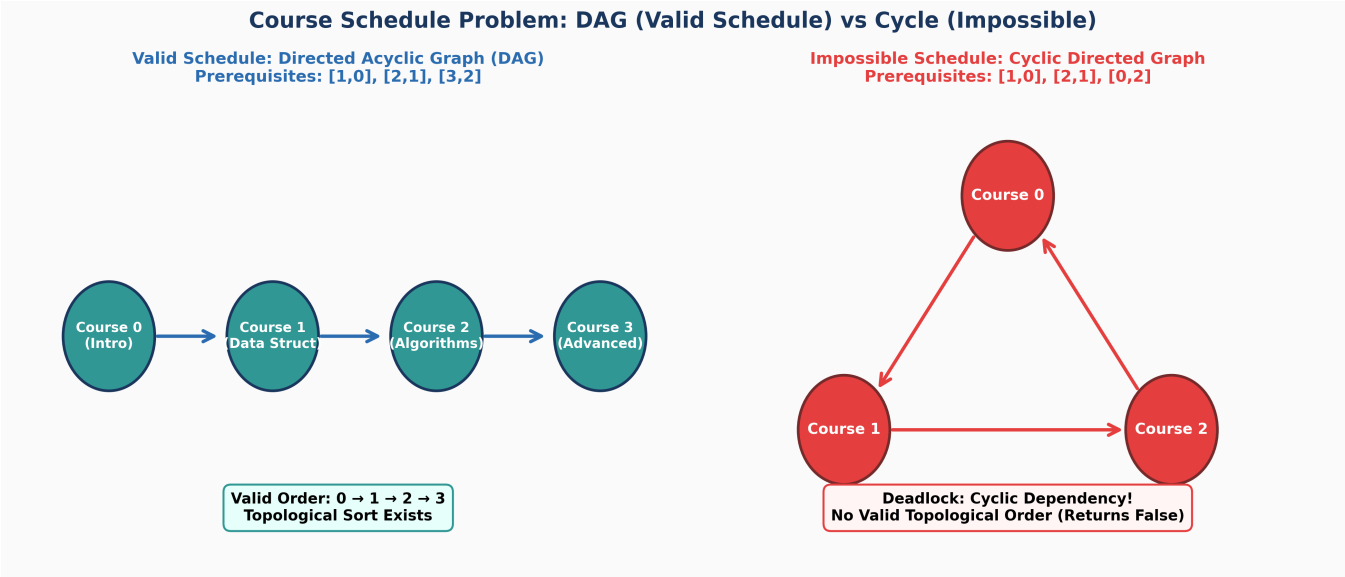


Figure 1: Valid Course Schedule DAG vs. Impossible Cyclic Schedule.

2. Kahn's Algorithm (BFS Indegree Approach)

Kahn's Algorithm systematically processes courses with an **indegree of 0** (no remaining prerequisites). As courses are completed, they are removed from the graph, reducing the indegree of dependent courses.

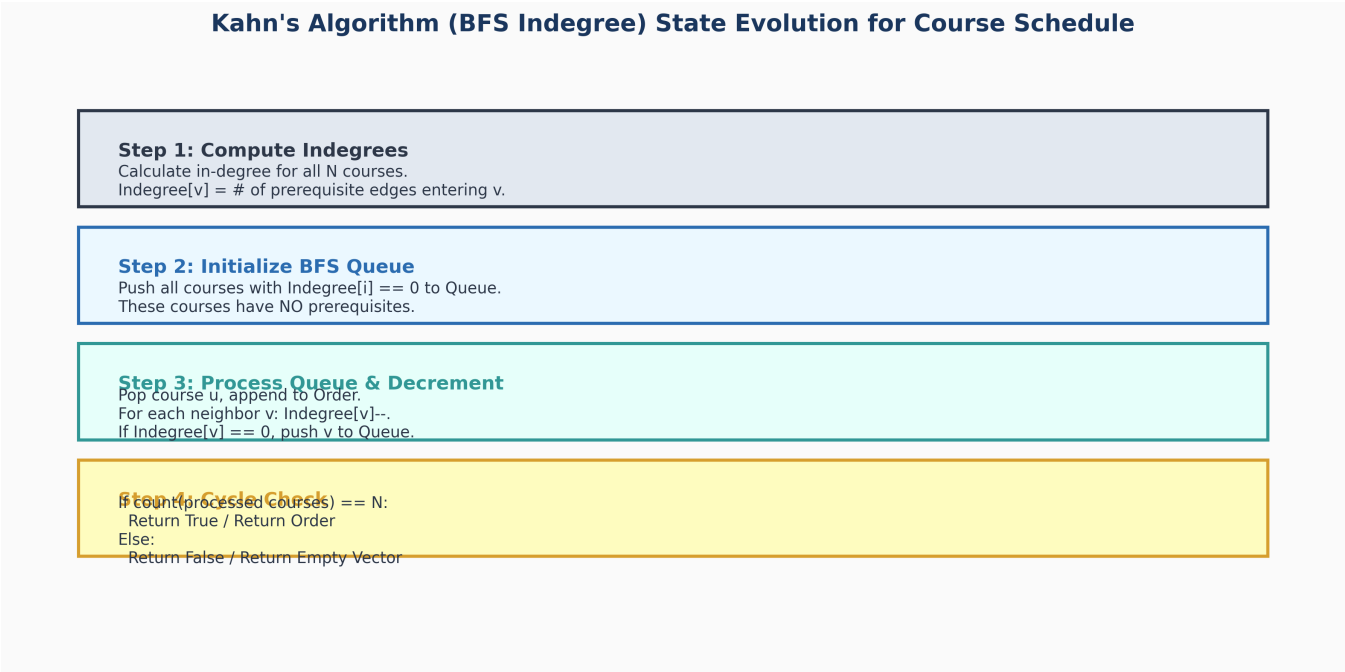


Figure 2: Kahn's Algorithm (BFS Indegree) State Evolution.

3. Production Object-Oriented C++ Implementation

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

class Solution {
public:
    // LeetCode 210: Returns Topological Sequence or Empty Vector if Cycle Exists
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> adj(numCourses);
        vector<int> indegree(numCourses, 0);

        for (const auto& pre : prerequisites) {
            int course = pre[0], prereq = pre[1];
            adj[prereq].push_back(course); // prereq -> course
            indegree[course]++;
        }

        queue<int> q;
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) q.push(i);
        }

        vector<int> topoOrder;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            topoOrder.push_back(u);
```

```
            for (int v : adj[u]) {
                indegree[v]--;
                if (indegree[v] == 0) q.push(v);
            }
        }

        return (topoOrder.size() == numCourses) ? topoOrder : vector<int>();
    }
};
```

4. Complexity Analysis & Technical Takeaways

Time Complexity: O(V + E) where V is numCourses and E is the number of prerequisites. Every vertex is queued once, and every directed edge is decremented once.
Space Complexity: O(V + E) for storing the adjacency list, queue, and indegree vector.
Key Takeaways:
• **Edge Direction:** Prerequisite [a, b] means b a.
• **Kahn's Algorithm vs DFS:** Kahn's algorithm is preferred in interviews because it handles both cycle detection and topological order generation cleanly without recursion stack risk.

Topological Sorting in Graph using DFS

Comprehensive Technical Guide, Stack Backtracking Mechanics & C++ Implementation

> **Topological Sorting** is a fundamental graph algorithm for **Directed Acyclic Graphs (DAGs)** that constructs a linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u always appears before vertex v in the sequence.

1. DAG Prerequisites & Linear Ordering Properties

For a graph to possess a valid topological ordering, it must strictly satisfy two conditions: it must be **Directed** (edges have an explicit orientation) and **Acyclic** (contains no directed cycles). If a graph contains a directed cycle (e.g., $u \rightarrow v \rightarrow w \rightarrow u$), a valid linear ordering is mathematically impossible because every node in the cycle depends on another.

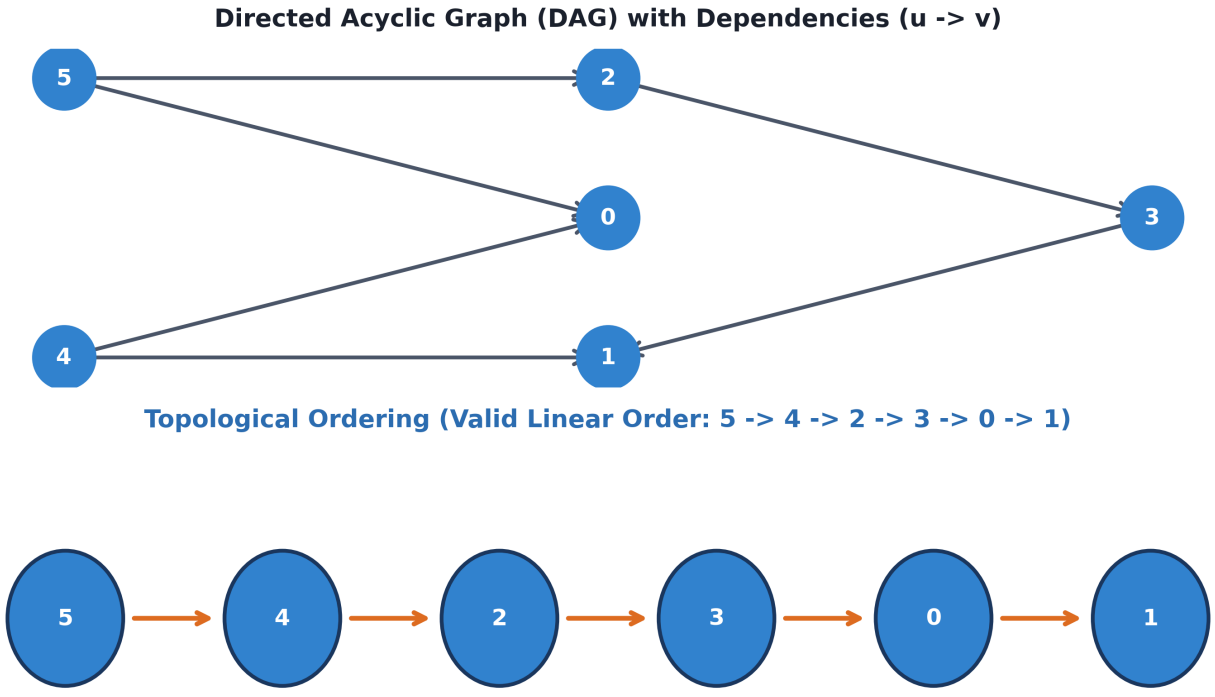


Figure 1: Directed Acyclic Graph (DAG) and its corresponding valid linear Topological Sort ordering.

2. DFS Backtracking & Stack Mechanics

The DFS-based topological sorting technique utilizes the post-order termination property of Depth-First Search. When visiting a vertex u, the algorithm recursively explores all of its adjacent neighbors first. Only when vertex u has no remaining unvisited adjacent neighbors (at the point of DFS function exit), vertex u is pushed onto a

LIFO **std::stack**. Once all vertices are processed, popping elements from the stack naturally yields a valid topological order.

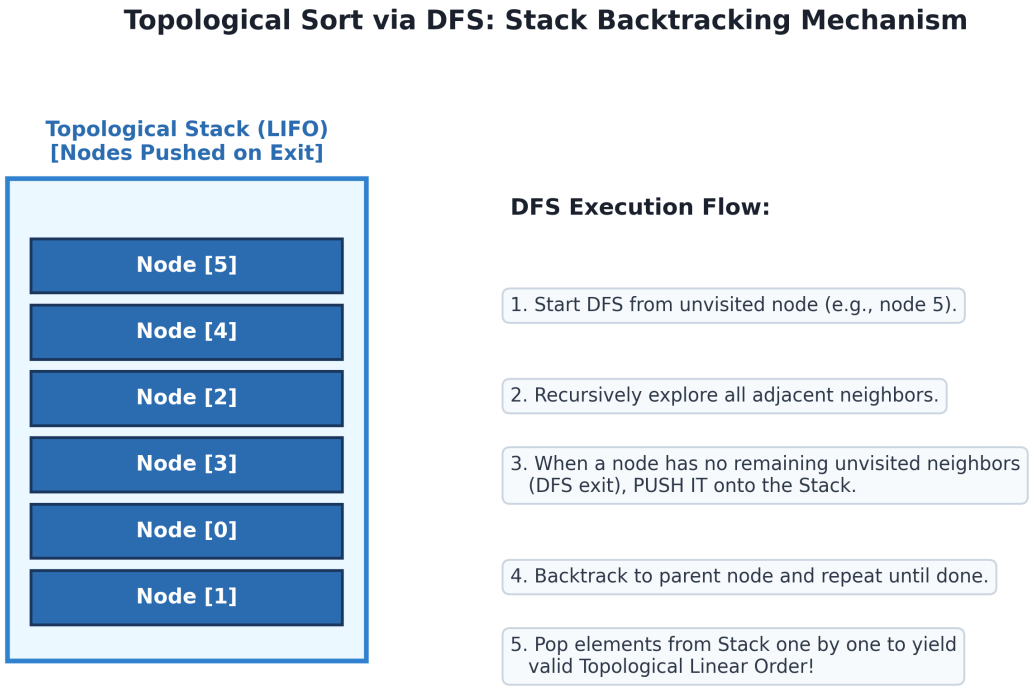


Figure 2: Stack Backtracking Mechanism pushing nodes on DFS exit.

3. Production C++ Implementation

```
#include <iostream>
#include <vector>
#include <list>
#include <stack>

using namespace std;

class Graph {
private:
    int V;
    list<int>* adj;

    void topoSortDFS(int node, vector<bool>& visited, stack<int>& st) {
        visited[node] = true;

        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                topoSortDFS(neighbor, visited, st);
            }
        }

        // Push to stack on DFS exit
        st.push(node);
    }
```

```
public:
    Graph(int vertices) {
        this->V = vertices;
        this->adj = new list<int>[V];
    }

    ~Graph() {
        delete[] adj;
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    vector<int> topologicalSort() {
        vector<bool> visited(V, false);
        stack<int> st;

        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                topoSortDFS(i, visited, st);
            }
        }

        vector<int> topoOrder;
        while (!st.empty()) {
            topoOrder.push_back(st.top());
            st.pop();
        }

        return topoOrder;
    }
};
```

4. Complexity Analysis & Technical Summary

Metric	Complexity	Algorithmic Explanation
Time Complexity	O(V + E)	Every vertex V is visited once, and every edge E is traversed once in adjacency list.
Space Complexity	O(V)	Auxiliary space for recursion call stack, boolean visited array, and result stack.

Detect a Cycle in Directed Graph Using DFS

Recursion Call Stack Traversal, Back-Edge Analysis & C++ Implementation

1. Core Problem: Undirected vs. Directed Cycle Detection

Cycle detection in **directed graphs** requires a fundamentally different approach than in undirected graphs. In an undirected graph, encountering any visited node that is not the direct parent triggers a cycle. However, in a directed graph, an edge from node A to node B does not allow movement from B to A. A node may be visited along a prior cross-path without creating a cycle.

> **Key Rule for Directed Graphs:** A cycle exists if and only if an edge points back to a vertex that is currently active in the **current recursion call stack path** (known as a **back-edge**).

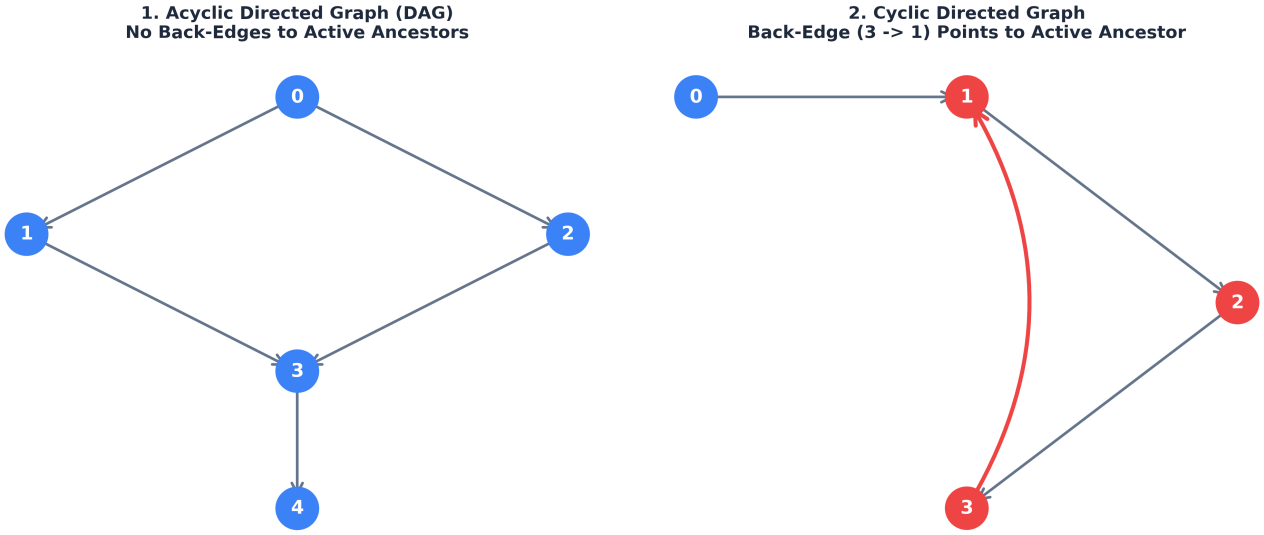


Figure 1: Contrast between Acyclic DAG and Cyclic Directed Graph with Back-Edge (3 -> 1)

2. Dual Array Mechanics: visited vs. pathVisited

To differentiate between nodes visited in completed paths versus nodes active in the current recursion stack, we utilize two state arrays:

• **visited (vis)**: Tracks whether a node has been encountered anywhere in the graph traversal.

• **pathVisited (pathVis / dfsVisited)**: Tracks whether a node is currently in the active recursion call stack.

When entering dfs(u), set both vis[u] = true and pathVis[u] = true. When exiting dfs(u) (backtracking), reset pathVis[u] = false.

DFS Cycle Detection Arrays: Visited vs. PathVisited (dfsVisited)

Vertex	Visited Array	PathVisited Array	DFS Recursion St
0	1 (True)	0 (Backtracked)	Explored & Popped from Stack
1	1 (True)	1 (Active)	Currently in Active DFS Path
2	1 (True)	1 (Active)	Currently in Active DFS Path
3	**1 (True)**	**1 (Active)**	**Cycle Trigger: Neighbor 1 is PathVisited=1!**

Figure 2: Execution State Trace of Visited vs. PathVisited Arrays during DFS

3. Production C++ Object-Oriented Code

```cpp
#include <iostream>
#include <vector>

using namespace std;

class DirectedGraph {
private:
    int V;
    vector<vector<int>> adj;

    bool dfsCheck(int node, vector<bool>& vis, vector<bool>& pathVis) {
        vis[node] = true;
        pathVis[node] = true;

        for (int neighbor : adj[node]) {
            if (!vis[neighbor]) {
                if (dfsCheck(neighbor, vis, pathVis)) return true;
            } else if (pathVis[neighbor]) {
                return true; // Back-edge detected!
            }
        }
        pathVis[node] = false; // Backtrack
        return false;
    }

public:
    DirectedGraph(int vertices) : V(vertices), adj(vertices) {}

    void addEdge(int u, int v) { adj[u].push_back(v); }

    bool isCyclic() {
        vector<bool> vis(V, false), pathVis(V, false);
        for (int i = 0; i < V; i++) {
            if (!vis[i]) {
                if (dfsCheck(i, vis, pathVis)) return true;
            }
        }
        return false;
    }
};
```

4. Time & Space Complexity

Metric	Complexity	Technical Explanation
Time Complexity	O(V + E)	Every node and directed edge is processed once during DFS.
Space Complexity	O(V)	Auxiliary vectors vis (O(V)) and pathVis (O(V)) plus recursion stack (O(V)).

Rotting Oranges (LeetCode 994): Multi-Source BFS Guide

Matrix Traversal, Level-Order Queue Mechanics & Object-Oriented C++ Implementation

1. Problem Overview & Graph Abstraction

The **Rotting Oranges** problem (LeetCode 994) models grid propagation where rot spreads 4-directionally (Up, Down, Left, Right) to fresh adjacent oranges every minute. We represent the $M \times N$ grid as an unweighted implicit graph where each matrix cell is a vertex V and valid 4-directional neighbor connections represent edges E with a uniform traversal cost of 1 minute.

> **Key Takeaway:** Multi-Source BFS generalizes single-source shortest path problems. When multiple rot sources exist at T = 0, enqueuing ALL rotten oranges initially ensures that every fresh orange is rotted along its optimal (shortest) path in a single linear O(M × N) pass.

2. Visual Grid Propagation Timeline & Paradigm Comparison

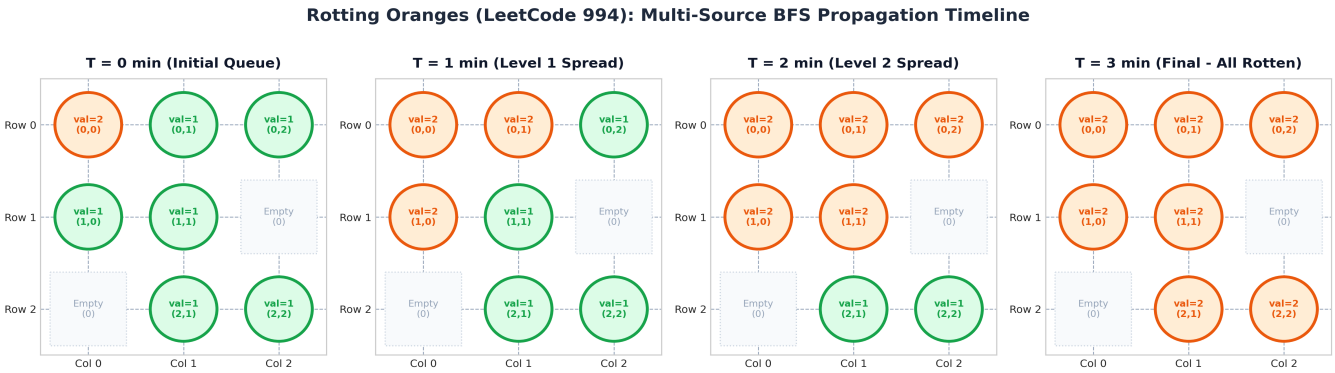


Figure 1: Grid state evolution across time steps T=0 to T=3 for a 3×3 matrix.

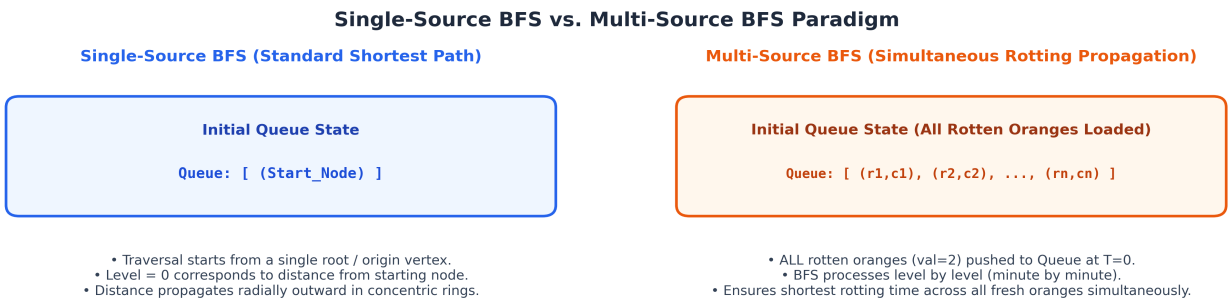


Figure 2: Architectural comparison between Single-Source and Multi-Source BFS initialization.

3. Step-by-Step State Trace Table

Time Step (T)	Queue Contents (Coordinates)	Cells Rotted in Current Level	Fresh Count
T = 0 min	[(0, 0)]	None (Initial multi-source load)	6
T = 1 min	[(0, 1), (1, 0)]	(0, 1) and (1, 0) marked rotten	4
T = 2 min	[(0, 2), (1, 1)]	(0, 2) and (1, 1) marked rotten	2
T = 3 min	[(2, 1)]	(2, 1) marked rotten	1
T = 4 min	[(2, 2)]	(2, 2) marked rotten	0

4. Production-Ready C++ Implementation

```
#include
#include
#include

using namespace std;

class Solution {
public:
    int orangesRotting(vector>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        queue> q;
        int freshCount = 0;

        // Step 1: Push all initial rotten oranges into Queue (Multi-Source Setup)
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == 2) q.push({r, c});
                else if (grid[r][c] == 1) freshCount++;
            }
        }

        if (freshCount == 0) return 0;

        int minutes = 0;
        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};

        // Step 2: Multi-Source Level-Order BFS
        while (!q.empty() && freshCount > 0) {
            int levelSize = q.size();
            minutes++;

            for (int i = 0; i < levelSize; i++) {
                auto [r, c] = q.front();
                q.pop();

                for (int d = 0; d < 4; d++) {
                    int nr = r + dr[d];
                    int nc = c + dc[d];

                    if (nr >= 0 && nr < m && nc >= 0 && nc < n && grid[nr][nc] == 1) {
                        grid[nr][nc] = 2; // In-place visited marking
                        freshCount--;
                        q.push({nr, nc});
                    }
                }
            }
        }

        return (freshCount == 0) ? minutes : -1;
    }
};
```

5. Complexity Analysis & Technical Takeaways

- **Time Complexity O(M × N):** Every grid cell is processed at most once during queue insertion and pop operations. 4-directional neighbor checks take O(1) constant time.
- **Space Complexity O(M × N):** The std::queue stores at most O(M × N) coordinate pairs in the worst case. In-place matrix modification (grid[nr][nc] = 2) eliminates the need for an extra visited matrix.
- **In-Place Visited Marking:** Marking cells rotten at the time of enqueuing prevents duplicate insertions into the queue.
- **Level-Wise Increment:** Time (minutes) is incremented per BFS level (iteration of levelSize), accurately simulating time ticks.

LeetCode 200: Number of Islands (C++)

Connected Components in 2D Matrix | Graph Algorithms

1. Graph Abstraction & Grid Representation

The **Number of Islands** problem (LeetCode 200) is a foundational algorithm question that demonstrates how to adapt graph traversal techniques (BFS/DFS) to 2D grid matrices. Although the grid is given as a 2D array of characters ('1' for land, '0' for water), it functions as an **implicit graph**.

Grid Matrix Concept	Graph Data Structure Equivalent
Cell grid[r][c] with '1'	Vertex / Node in the Graph
Adjacent cells (Up, Down, Left, Right)	Undirected Edges connecting neighbor vertices
Isolated group of connected '1's	Connected Component in an undirected graph
Total Number of Islands	Total Number of Connected Components

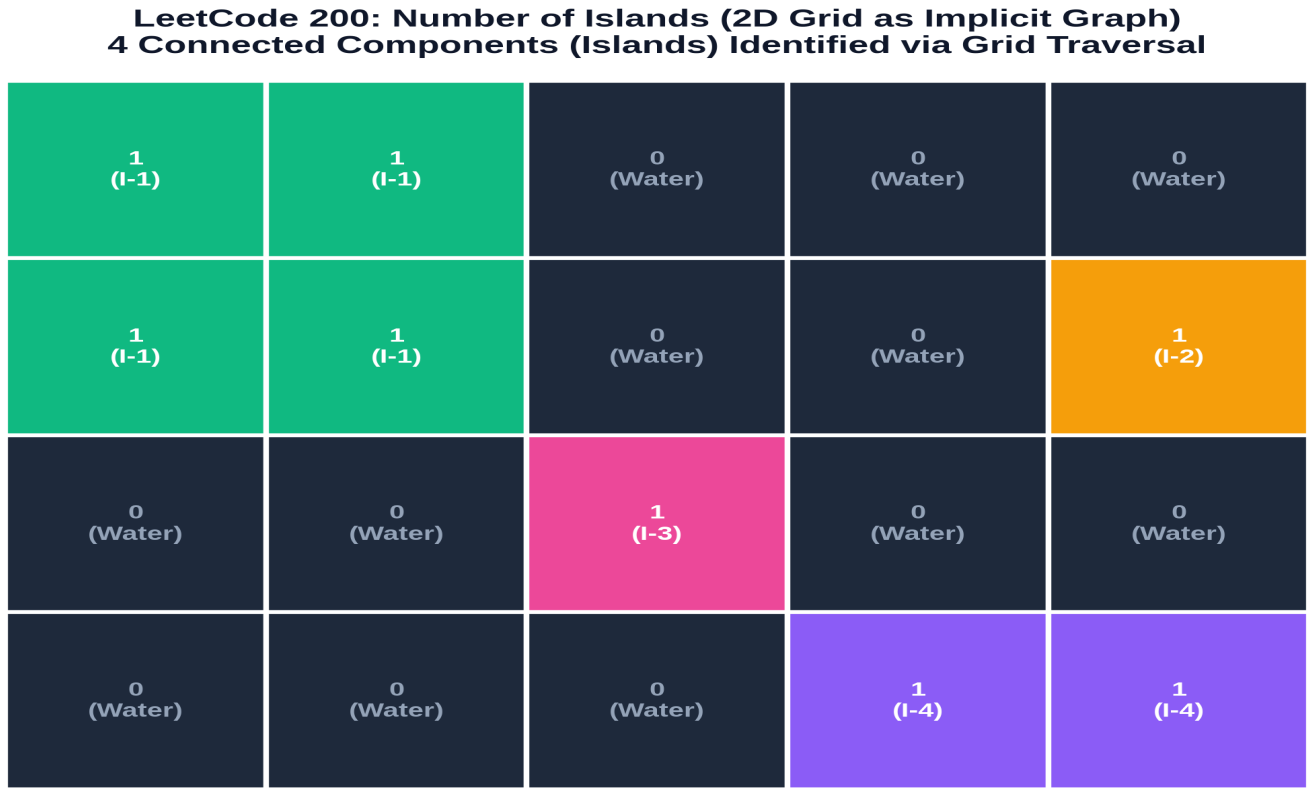


Figure 1: 2D Grid with 4 Connected Components (Islands) identified via Grid Traversal.

2. 4-Directional Traversal Vectors & Bounds Checking

To explore neighbors in a 2D grid, we check the 4 cardinal directions (Up, Down, Left, Right). We define direction vectors dr = {-1, 1, 0, 0} and dc = {0, 0, -1, 1} to iterate cleanly over all neighbors.

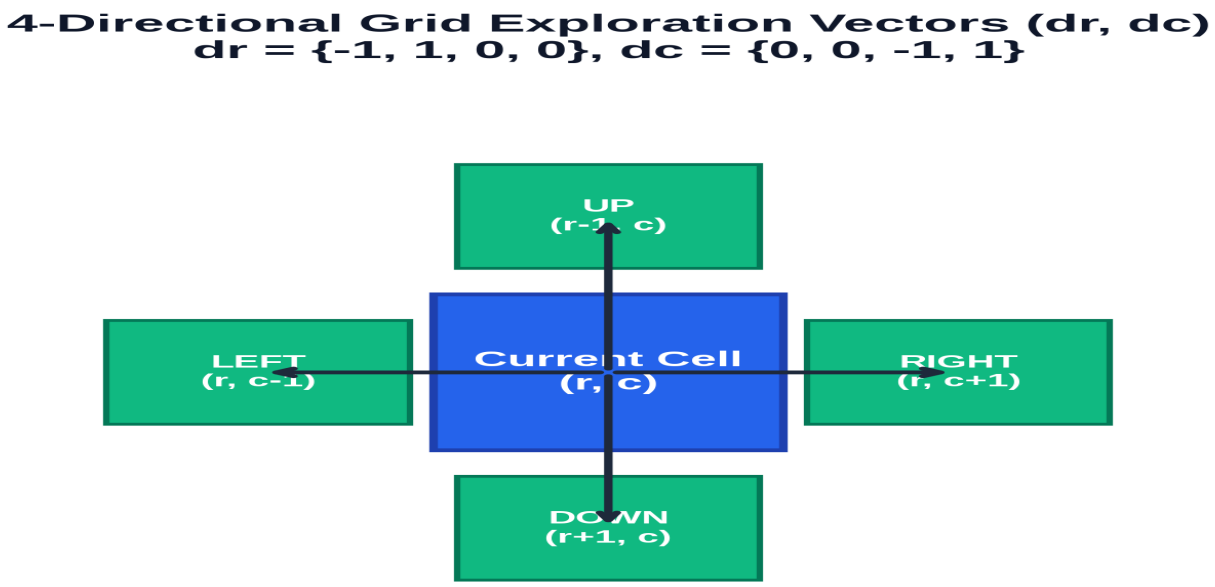


Figure 2: 4-Directional Grid Exploration Vectors (dr, dc).

3. C++ BFS & DFS Implementations

```
// Method 1: Breadth-First Search (BFS Queue)
int numIslands(vector>& grid) {
    int m = grid.size(), n = grid[0].size(), islands = 0;
    int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
    for (int r = 0; r < m; ++r) {
        for (int c = 0; c < n; ++c) {
            if (grid[r][c] == '1') {
                islands++;
                grid[r][c] = '0'; // Sink land (mark visited)
                queue> q;
                q.push({r, c});
                while (!q.empty()) {
                    auto [curr_r, curr_c] = q.front(); q.pop();
                    for (int i = 0; i < 4; ++i) {
                        int nr = curr_r + dr[i], nc = curr_c + dc[i];
                        if (nr >= 0 && nr < m && nc >= 0 && nc < n && grid[nr][nc] == '1') {
                            grid[nr][nc] = '0'; // Mark before push
                            q.push({nr, nc});
                        }
                    }
                }
            }
        }
    }
    return islands;
}
```

```
// Method 2: Depth-First Search (DFS Recursion)
void dfs(vector>& grid, int r, int c, int m, int n) {
    if (r < 0 || r >= m || c < 0 || c >= n || grid[r][c] == '0') return;
    grid[r][c] = '0'; // Sink land
    dfs(grid, r - 1, c, m, n); // Up
```

```
    dfs(grid, r + 1, c, m, n); // Down
    dfs(grid, r, c - 1, m, n); // Left
    dfs(grid, r, c + 1, m, n); // Right
}
```

4. Time & Space Complexity Analysis

Metric	Complexity	Explanation
Time Complexity	O(M * N)	Every cell in the M x N matrix is visited at most a constant number of times.
Space Complexity (BFS)	O(min(M, N))	Queue size is bounded by the diagonal length of the grid.
Space Complexity (DFS)	O(M * N)	Worst-case recursion call stack depth when grid is filled with land.

> **Technical Interview Key Takeaways:**
> 1. **Sinking Islands Trick:** Mutating grid[r][c] = '0' avoids allocating extra O(M x N) space for a visited matrix.
> 2. **Enqueue Marking in BFS:** Always mark cells as visited ('0') *immediately when pushing into the queue* to prevent redundant queue entries.
> 3. **Direction Arrays:** Using dr[] and dc[] keeps neighbor traversal code clean and bug-free.

Detect a Cycle in an Undirected Graph using BFS

Level-Order Traversal Strategy, Parent Pointer Check, Queue Mechanics & C++ Implementation

1. Algorithmic Intuition & The Parent Pointer Rule

In an undirected graph, every edge connecting vertex **u** and vertex **v** represents a bidirectional connection (u v). When traversing from vertex u to vertex v, vertex v will immediately see vertex u in its adjacency list. Therefore, simply marking nodes as *visited* is insufficient—we must differentiate between returning to an immediate parent and discovering a true cycle.

> **The Core Cycle Decision Rule:**
> When popping {u, parent} from the queue and inspecting adjacent neighbor v:
> • **Case 1 (Unvisited)**: Mark visited[v] = true, push {v, u} into queue.
> • **Case 2 (Visited AND v == parent)**: Trivial reverse edge back to immediate predecessor. Ignore.
> • **Case 3 (Visited AND v != parent)**: **CYCLE DETECTED!** An alternate path has reached vertex v.

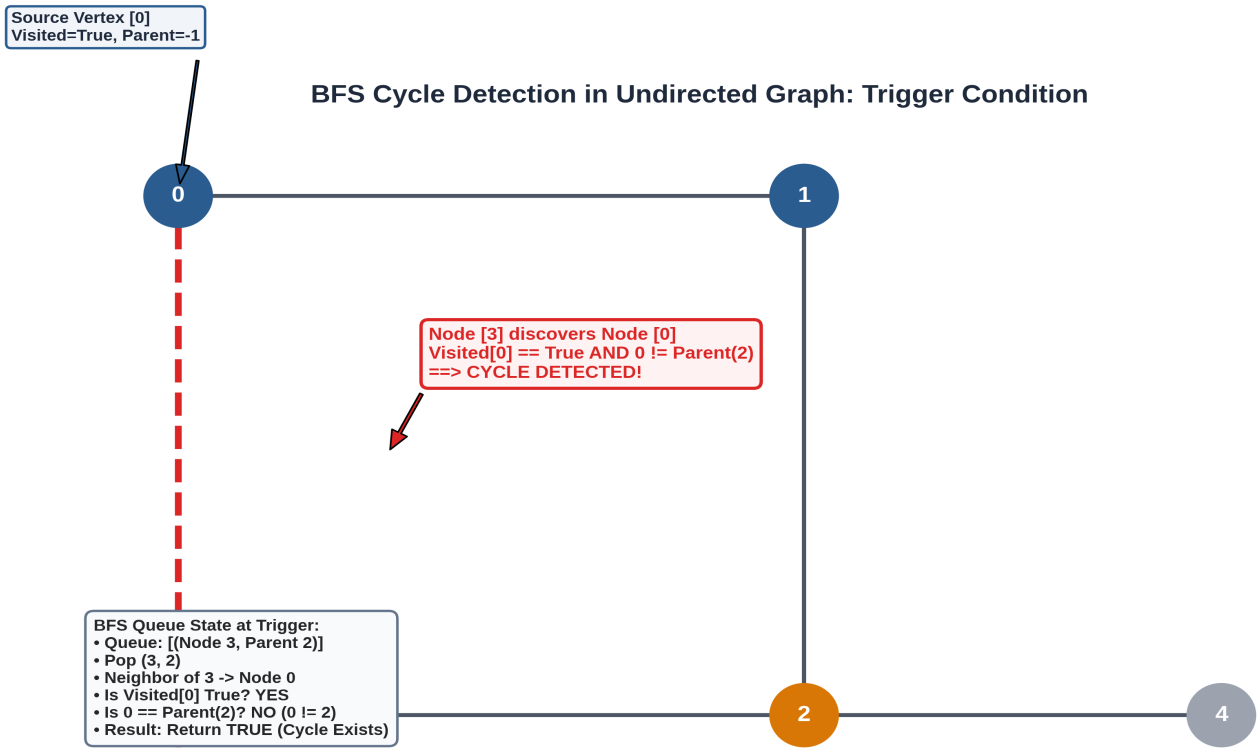


Figure 1: BFS Cycle Trigger Condition when Node 3 discovers Node 0 (already visited and 0 != Parent 2).

2. Production-Ready C++ Implementation

```
#include <iostream>
#include <vector>
#include <list>
#include <queue>

using namespace std;

class Graph {
private:
    int V;
    list<int>* adj;

    bool bfsCheckCycle(int startNode, vector<bool>& visited) {
        queue<pair<int, int>> q; // {currentNode, parentNode}
        visited[startNode] = true;
        q.push({startNode, -1});

        while (!q.empty()) {
            int node = q.front().first;
            int parent = q.front().second;
            q.pop();

            for (int neighbor : adj[node]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push({neighbor, node});
                } else if (neighbor != parent) {
                    return true; // Cycle detected!
                }
            }
        }
        return false;
    }

public:
    Graph(int V) { this->V = V; adj = new list<int>[V]; }
    ~Graph() { delete[] adj; }
    void addEdge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }

    bool isCyclic() {
        vector<bool> visited(V, false);
        for (int i = 0; i < V; ++i) {
            if (!visited[i] && bfsCheckCycle(i, visited)) return true;
        }
        return false;
    }
};
```

3. Algorithm Execution Trace Table

Trace for graph with edges (0-1), (1-2), (2-3), (3-0), (2-4):

Step	Popped {node, parent}	Neighbor	Visited?	Condition Evaluation	Action
1	{0, -1}	1	False	!visited[1] Mark Visited	Push {1, 0}
2	{0, -1}	3	False	!visited[3] Mark Visited	Push {3, 0}

Step	Popped {node, parent}	Neighbor	Visited?	Condition Evaluation	Action
3	{1, 0}	0	True	0 == parent (0 == 0)	Ignore (Parent edge)
4	{1, 0}	2	False	!visited[2] Mark Visited	Push {2, 1}
5	{3, 0}	0	True	0 == parent (0 == 0)	Ignore (Parent edge)
6	{3, 0}	2	True	2 != parent (2 != 0)	Already visited/enqueued
7	{2, 1}	3	True	3 != parent (3 != 1)	**CYCLE DETECTED! Return True**

4. Comparative Analysis: BFS vs. DFS Cycle Detection

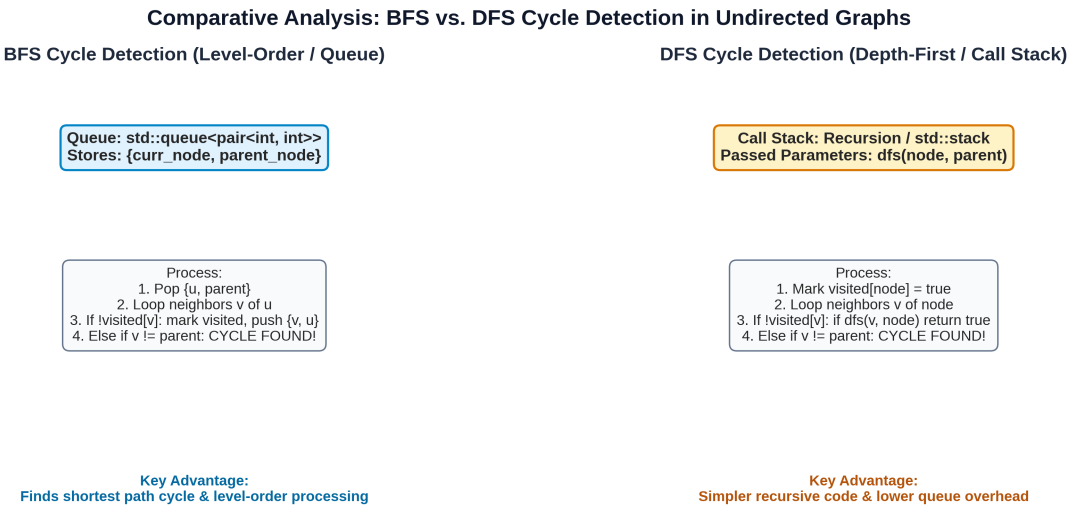


Figure 2: Architectural comparison between BFS queue-based and DFS call-stack-based cycle detection.

5. Complexity Summary & Key Takeaways

Metric	Complexity	Technical Justification
Time Complexity	O(V + E)	Every vertex is enqueued/dequeued once; every adjacency edge is traversed twice.
Space Complexity	O(V)	Visited vector of size V and queue size up to V in worst-case.
Disconnected Component Handling	Loop 0...V-1	Outer loop ensures unvisited vertices in isolated components trigger BFS.

Cycle Detection in Undirected Graphs Using DFS

Comprehensive C++ Architecture, Parent Tracking Mechanics & Walkthrough

1. Core Paradigm: Parent Tracking Mechanism

Detecting cycles in an undirected graph is a fundamental problem in graph theory and algorithm design. In an undirected graph, every edge connecting vertex **u** and vertex **v** represents a bidirectional connection. Because of this symmetry, moving from **u** to **v** naturally exposes **u** as a neighbor of **v**. Therefore, standard visitation checks alone are insufficient — we must explicitly track the **parent vertex** that led to the current node during Depth-First Search (DFS) traversal.

> **The Fundamental Condition:** A cycle exists if and only if DFS encounters a neighbor node **nbr** that is ALREADY VISITED and IS NOT the immediate parent node (visited[nbr] == true && nbr != parent).

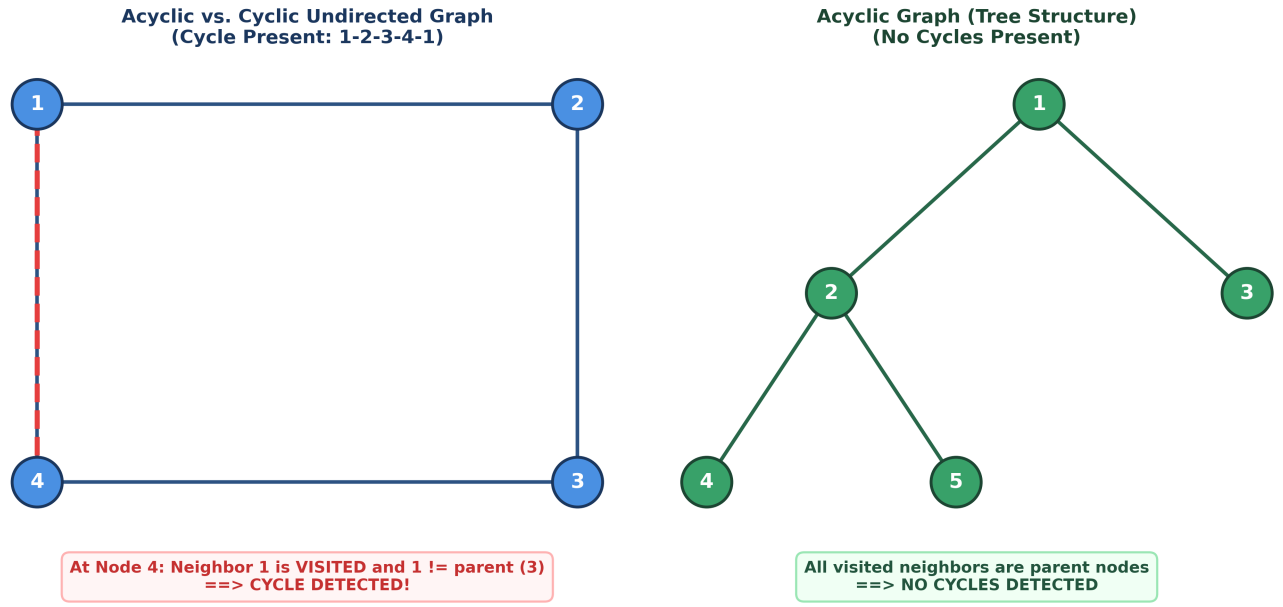


Figure 1: Structural comparison between a cyclic undirected graph and an acyclic tree structure.

2. Execution Trace & Recursion Call Stack

To visualize how parent tracking prevents false positives while successfully flagging real cycles, consider a 4-vertex ring graph with edges (1, 2), (2, 3), (3, 4), and (4, 1).

Step	Curr Node	Parent	Visited Array	Neighbor Check	Action / Decision
1	1	-1	{1:T, 2:F, 3:F, 4:F}	2 (unvisited)	Recurse DFS(2, parent=1)

2	2	1	{1:T, 2:T, 3:F, 4:F}	1 (visited, is parent) 3 (unvisited)	Skip trivial back-edge Recurse DFS(3, parent=2)
3	3	2	{1:T, 2:T, 3:T, 4:F}	2 (visited, is parent) 4 (unvisited)	Skip trivial back-edge Recurse DFS(4, parent=3)
4	4	3	{1:T, 2:T, 3:T, 4:T}	3 (visited, is parent) **1 (visited, NOT parent!)**	**visited[1] && 1!=3 ==> RETURN TRUE!**

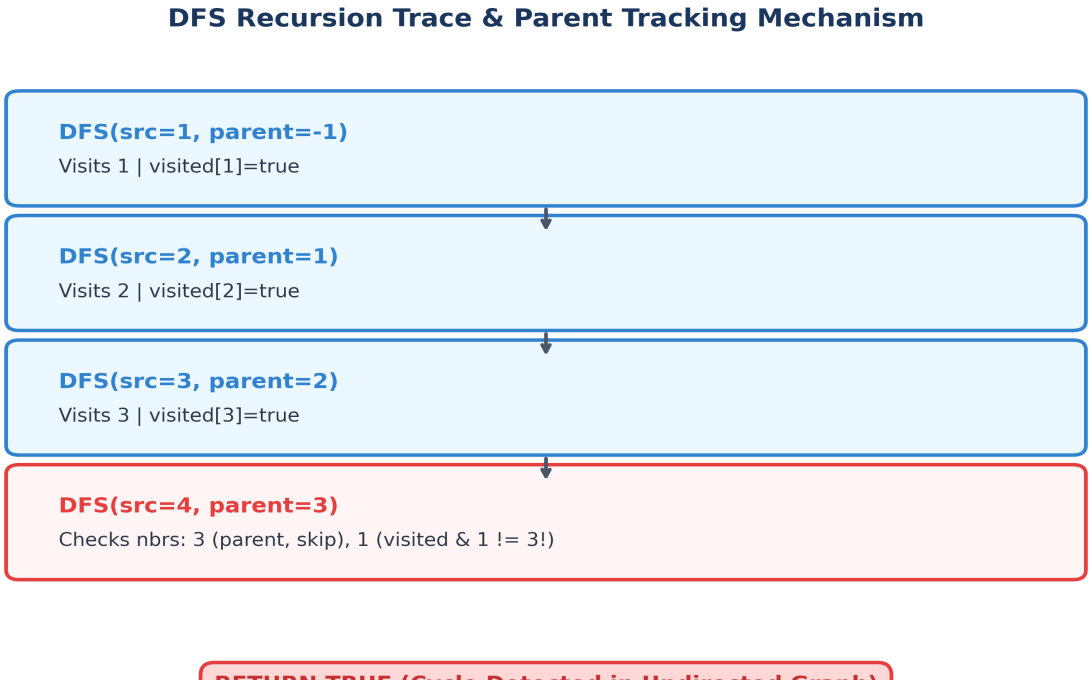


Figure 2: DFS Call Stack state evolution and cycle detection trigger.

3. Production C++ Implementation

```
#include <iostream>
#include <vector>
using namespace std;
class GraphCycleDetector {
private:
    int V;
    vector<vector<int>> adj;
    bool dfsCycle(int curr, int parent, vector<bool>& visited) {
        visited[curr] = true;
        for (int nbr : adj[curr]) {
            if (!visited[nbr]) {
                if (dfsCycle(nbr, curr, visited)) return true;
            }
            else if (nbr != parent) {
```

```
                return true; // Cycle detected!
            }
        }
        return false;
    }
public:
    GraphCycleDetector(int vertices) : V(vertices), adj(vertices) {}
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    bool hasCycle() {
        vector<bool> visited(V, false);
        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                if (dfsCycle(i, -1, visited)) return true;
            }
        }
        return false;
    }
};
```

4. Complexity Analysis & Takeaways

Time Complexity: O(V + E) — Every vertex is marked visited once, and every edge is examined at most twice.

Space Complexity: O(V) — Requires O(V) space for the visited array and recursion stack.

Depth-First Search (DFS) Traversal in C++

A Rigorous Exploration of Stack Mechanics, Recursion, Disconnected Graphs & Algorithm Complexity

> **Core Technical Takeaway:** Depth-First Search (DFS) traverses graphs by exploring as far as possible along each branch before backtracking. Driven natively by the call stack (LIFO) or explicit std::stack, DFS achieves **O(V + E)** time complexity and **O(V)** auxiliary space complexity.

1. Theoretical Foundations & Call Stack Mechanics

Depth-First Search explores graph nodes deep into a path before visiting siblings or backtracking. To avoid infinite loops caused by cycles, a boolean vector visited[u] tracks processed vertices.

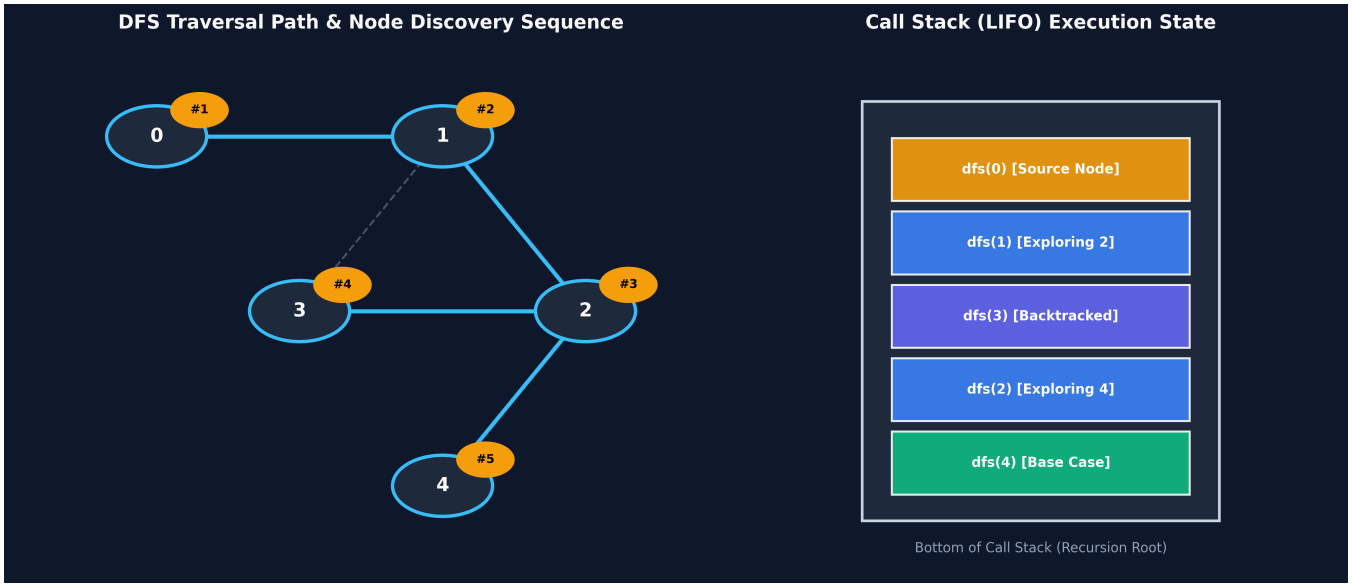


Figure 1: DFS Traversal Trace & Recursive Call Stack Execution Model

2. Step-by-Step Traversal Walkthrough

Step	Node (u)	Visited State	Call Stack State (Top -> Bottom)	Transition / Action
1	0	[T, F, F, F, F]	[0]	Start at source 0; mark visited; explore neighbor 1.
2	1	[T, T, F, F, F]	[1, 0]	Recurse to 1; mark visited; explore neighbor 2.
3	2	[T, T, T, F, F]	[2, 1, 0]	Recurse to 2; mark visited; explore neighbor 3.
4	3	[T, T, T, T, F]	[3, 2, 1, 0]	Recurse to 3; mark visited. Neighbors processed -> Backtrack.

Step	Node (u)	Visited State	Call Stack State (Top -> Bottom)	Transition / Action
5	2	[T, T, T, T, F]	[2, 1, 0]	Resume 2; explore unvisited neighbor 4.
6	4	[T, T, T, T, T]	[4, 2, 1, 0]	Recurse to 4; mark visited. No unvisited neighbors -> Backtrack.
7	2	[T, T, T, T, T]	[2, 1, 0]	All neighbors of 2 fully explored -> Backtrack to 1.
8	1	[T, T, T, T, T]	[1, 0]	Neighbor 3 already visited -> Backtrack to 0.
9	0	[T, T, T, T, T]	[0]	Call stack empties -> Traversal Complete.

3. Production-Ready C++ Implementation

The C++ implementation utilizes an Adjacency List (std::list<int>* l) with dynamic memory allocation. A private dfsHelper handles recursive depth-first exploration while the outer dfs guarantees complete coverage of disconnected graphs.

```cpp
#include <iostream>
#include <list>
#include <vector>

using namespace std;

class Graph {
private:
    int V;
    list<int>* l;

    void dfsHelper(int u, vector<bool>& visited) {
        visited[u] = true;
        cout << u << " ";

        for (int neighbor : l[u]) {
            if (!visited[neighbor]) {
                dfsHelper(neighbor, visited);
            }
        }
    }

public:
    Graph(int vertices) {
        this->V = vertices;
        this->l = new list<int>[V];
    }

    ~Graph() {
        delete[] l;
    }

    void addEdge(int u, int v, bool isDirected = false) {
        l[u].push_back(v);
        if (!isDirected) {
            l[v].push_back(u);
        }
    }

    void dfs(int startNode) {
        vector<bool> visited(V, false);
        cout << "DFS Traversal: ";
        dfsHelper(startNode, visited);

        // Handle Disconnected Components
        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                dfsHelper(i, visited);
            }
        }
        cout << endl;
    }
};
```

4. Comparative Analysis: BFS vs. DFS Paradigm

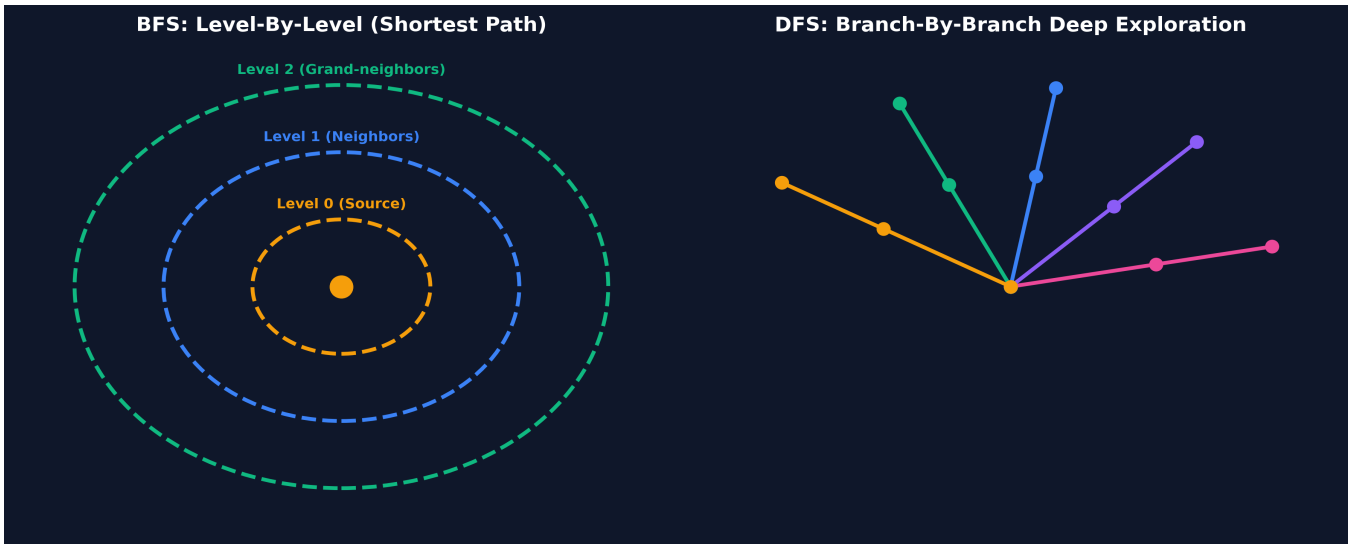


Figure 2: BFS Concentric Level Expansion vs. DFS Deep Path Exploration

Dimension	Breadth-First Search (BFS)	Depth-First Search (DFS)
Data Structure	Queue (std::queue) - FIFO	Call Stack / std::stack - LIFO
Exploration Style	Concentric Level-by-Level	Branch-by-Branch Deep Dive
Shortest Path Guarantee	Yes (for unweighted graphs)	No (explores paths arbitrarily)
Memory Complexity	O(W) where W = Max Graph Width	O(H) where H = Max Graph Depth
Primary Use Cases	Shortest paths, Peer-to-Peer Networks	Cycle Detection, Topological Sort, Mazes
Time Complexity	O(V + E)	O(V + E)

Breadth-First Search (BFS) in C++

Algorithm Architecture, FIFO Queue Mechanics, Level-Order Paradigm & Code Walkthrough

1. Fundamental Concepts & Level-Order Paradigm

Breadth-First Search (BFS) is a fundamental graph traversal algorithm that explores vertices level-by-level radiating outward from a designated source node. Unlike hierarchical trees, general graphs contain arbitrary connections, multiple candidate paths, and potential cycles. BFS systematically guarantees that all vertices at distance *k* from the source are processed before any vertex at distance *k+1* is expanded. Consequently, BFS inherently discovers the **shortest path in unweighted graphs**.

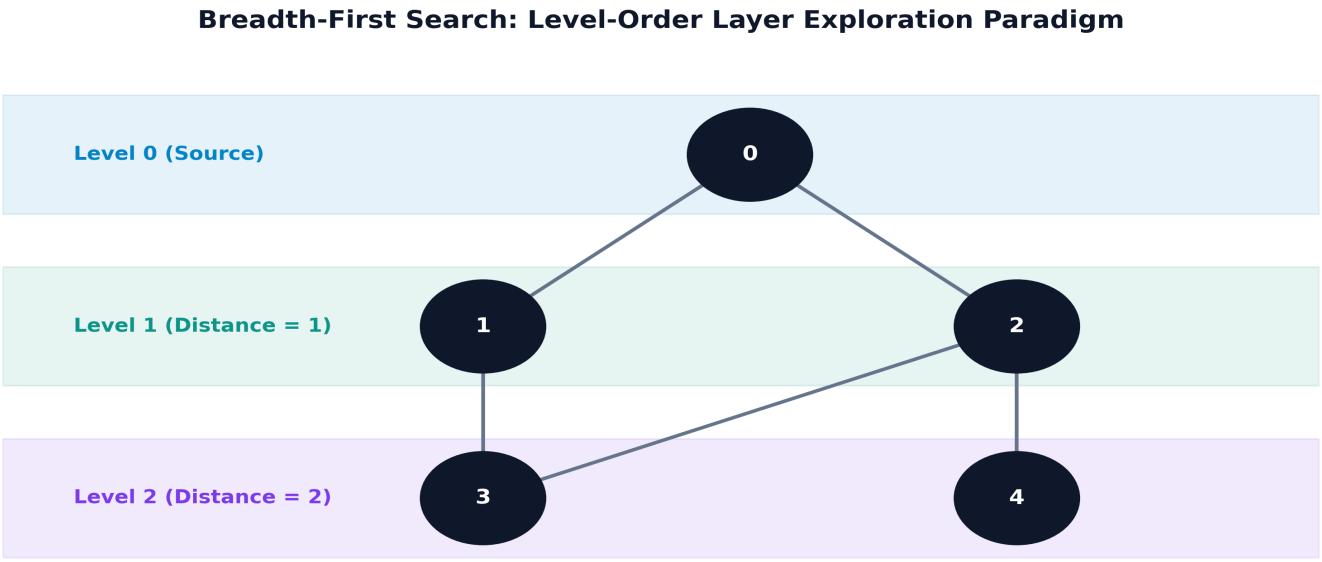


Figure 1: Concentric level-order expansion during Breadth-First Traversal.

2. Auxiliary Data Structures

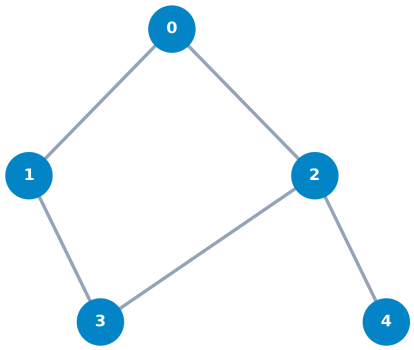
To achieve deterministic level-order traversal, BFS relies on two essential auxiliary containers:
• **FIFO Queue (std::queue<int>)**: Maintains the frontier of discovered vertices. Nodes at level *k* are dequeued first from the front, while their unvisited neighbors (level *k+1*) are enqueued at the back.
• **Visited Tracking Vector (std::vector<bool>)**: Tracks whether a vertex has already been enqueued. This safeguards against duplicate node processing and infinite recursion loops in cyclic or undirected graphs.

CRITICAL IMPLEMENTATION PITFALL

Enqueuing Rule: A node must be marked as *visited* = *true* immediately when it is **pushed into the queue**, rather than when it is popped. Marking nodes upon popping leads to duplicate queue entries and exponential memory buildup.

Consider a graph with $V = 5$ vertices (labeled 0 to 4) and edges $(0,1), (0,2), (1,3), (2,3), (2,4)$. Starting BFS at vertex 0 yields the step-by-step queue transitions and visited array updates detailed below:

Sample Graph & BFS Traversal Order



BFS Traversal Sequence:
0 → 1 → 2 → 3 → 4

Queue (FIFO) & Visited Array Trace

Step	Pop Node	Push Neighbors to Queue	Queue State (FIFO)	Visited Array
1	Init (0)	1, 2	[1, 2]	[T, T, T, F, F]
2	Pop 1	3	[2, 3]	[T, T, T, T, F]
3	Pop 2	4	[3, 4]	[T, T, T, T, T]
4	Pop 3	None (already visited)	[4]	[T, T, T, T, T]
5	Pop 4	None	[Empty]	[T, T, T, T, T]

Figure 2: Complete step-by-step state trace of FIFO queue and visited boolean tracker.

Step	Popped Node	Neighbors Examined	Enqueued Neighbors	Queue State (FIFO)	Visited Vector State
0	Init	Source Node 0	0	[0]	[T, F, F, F, F]
1	0	1, 2	1, 2	[1, 2]	[T, T, T, F, F]
2	1	0 (visited), 3	3	[2, 3]	[T, T, T, T, F]
3	2	0 (v), 3 (v), 4	4	[3, 4]	[T, T, T, T, T]
4	3	1 (v), 2 (v)	None	[4]	[T, T, T, T, T]
5	4	2 (v)	None	[] (Empty)	[T, T, T, T, T]

Resulting Traversal Order: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

```cpp
// Production-grade Object-Oriented C++ Implementation for BFS Traversal
#include <iostream>
#include <list>
#include <vector>
#include <queue>

using namespace std;

class Graph {
private:
    int V;                  // Total number of vertices
    list<int>* l;          // Dynamic array of neighbor lists

public:
    Graph(int vertices) {
        this->V = vertices;
        this->l = new list<int>[V];
    }

    ~Graph() {
        delete[] l;
    }

    void addEdge(int u, int v, bool isDirected = false) {
        l[u].push_back(v);
        if (!isDirected) l[v].push_back(u);
    }

    // Single Source BFS Traversal
    void bfs(int src) const {
        vector<bool> visited(V, false);
        queue<int> q;

        visited[src] = true;
        q.push(src);

        cout << "BFS Traversal Order: ";
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            cout << curr << " ";

            for (int nbr : l[curr]) {
                if (!visited[nbr]) {
                    visited[nbr] = true; // Mark visited upon push
                    q.push(nbr);
                }
            }
        }
        cout << endl;
    }
};
```

Real-world graphs frequently contain isolated components or disconnected sub-graphs. To ensure 100% vertex coverage across all components, the main function iterates over all vertices 0 through V-1, triggering BFS whenever an unvisited vertex is encountered.

```
// Handling Disconnected Graph Components
void bfsDisconnected() const {
    vector<bool> visited(V, false);
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            queue<int> q;
            visited[i] = true;
            q.push(i);
            while (!q.empty()) {
                int curr = q.front(); q.pop();
                cout << curr << " ";
                for (int nbr : l[curr]) {
                    if (!visited[nbr]) {
                        visited[nbr] = true;
                        q.push(nbr);
                    }
                }
            }
        }
    }
}
```

6. Asymptotic Complexity Proofs

Resource Dimension	Complexity	Mathematical Rigor & Derivation
Time Complexity	O(V + E)	Every vertex is enqueued/dequeued exactly once (O(V)). For every dequeued vertex, its adjacency list is iterated over. The total list iterations sum to 2E (undirected) or E (directed).
Space Complexity	O(V)	The FIFO queue holds at most O(V) nodes during peak level width. The visited boolean vector requires O(V) auxiliary memory.

TECHNICAL INTERVIEW SUMMARY

1. **Shortest Path Guarantee:** BFS guarantees the shortest path (fewest edges) in unweighted graphs.
2. **FIFO Queue vs. LIFO Stack:** Replacing std::queue with std::stack converts BFS into Depth-First Search (DFS).
3. **Memory Hygiene:** Always implement class destructors (delete[] l) when allocating dynamic adjacency list arrays.

Graph Data Structures in C++

Comprehensive Guide, Architecture, Memory Mechanics & Implementation

1. Introduction to Graph Data Structures & Foundations

At a fundamental level, a graph is a non-linear data structure comprising a finite set of vertices (nodes) interconnected by a set of edges (connections). Mathematically, a graph *G* is defined as an ordered pair ***G* = (*V*, *E*)**, where *V* represents the set of vertices and *E* represents the set of edges connecting pairs of vertices. Unlike linear data structures such as arrays or linked lists, graphs allow arbitrary relationships between elements without requiring a sequential ordering.

1.1 Terminology: Vertices vs. Edges

- **Vertex (*V*)**: Represents a discrete entity, such as a user profile in a social network, a geographical location on a map, or a system state in game engines.
- **Edge (*E*)**: Represents a relationship, pathway, or communication link between two vertices.

1.2 Structural Comparison: Binary Trees vs. Graphs

To understand graphs, it is instructive to compare them with binary trees. A tree is a specialized, restricted subset of a graph enforcing strict hierarchical rules. The table below highlights their operational differences:

Structural Attribute	Binary Tree	Graph Data Structure
Hierarchy	Strict Parent-Child hierarchy with root node	No hierarchy; peer-to-peer network
Path Uniqueness	Exactly one unique path from root to any node	Multiple paths or no paths between vertices
Cycles	Strictly acyclic (no loops allowed)	Can be cyclic or acyclic
Node Connections	At most 1 parent; at most 2 children per node	Arbitrary number of incoming/outgoing edges
Root Node	Requires a distinct starting root node	No designated root node natively

2. Real-World Engineering Applications

Graphs are foundational across software engineering because real-world data inherently forms interconnected networks rather than simple linear lists.

- **Geographic Routing & Navigation:** Navigation systems like Google Maps model intersections as vertices and roads as weighted edges, finding optimal paths using shortest-path algorithms.
- **Social Network Architecture:** Platforms like Facebook and LinkedIn model profiles as vertices and connections as edges (undirected for mutual friendships, directed for follows).
- **Neural Networks & Machine Learning:** Deep learning frameworks structure computational models as directed graphs where nodes represent artificial neurons and edges carry synaptic weights.

Key Insight: Graphs break hierarchical constraints, allowing software engineers to model complex real-world relationships with dynamic connectivity.

3. Taxonomy & Classifications of Graphs

Graphs are categorized based on edge directionality, edge weights, and component connectivity.

3.1 Edge Directionality & Weighting

1. **Undirected Graphs:** Edges represent bidirectional links (traversal permitted $u \to v$ and $v \to u$).
2. **Directed Graphs (Digraphs):** Edges possess explicit directionality (traversal permitted strictly along arrow direction).
3. **Weighted Graphs:** Edges carry numerical values $w \in \mathbb{R}$ representing distance, latency, or cost.
4. **Unweighted Graphs:** All edges carry uniform traversal cost.

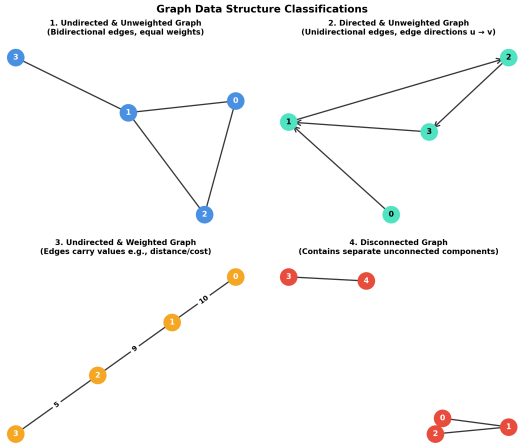


Figure 1: Visual comparison of Graph Classifications (Undirected vs. Directed, Unweighted vs. Weighted).

3.2 Four-Way Classification Matrix

Classification	Edge Characteristics	Real-World Analogy
Undirected & Unweighted	Bidirectional; uniform cost	Standard Facebook friendship networks
Undirected & Weighted	Bidirectional; explicit numerical values	Two-way road networks with physical distance
Directed & Unweighted	Unidirectional; uniform cost	Twitter/Instagram follow networks
Directed & Weighted	Unidirectional; explicit numerical values	One-way toll roads or flight pricing paths

4. Graph Representation Techniques in C++

To programmatically operate on graphs, vertex and edge data must be stored efficiently in memory. The two primary techniques are the **Adjacency Matrix** and the **Adjacency List**.

Metric / Dimension	Adjacency Matrix	Adjacency List
Space Complexity	$O(V^2)$ (Fixed 2D array size)	$O(V + E)$ (Optimal for sparse graphs)
Edge Lookup Time	$O(1)$ (Direct indexing matrix[u][v])	$O(\text{degree}(u))$ (Scans list at vertex u)

Find All Neighbors	***O*(*V*)** (Scans entire row)	***O*(degree(u))** (Iterates existing edges only)
Memory Allocation	Rigid allocation; wasteful for sparse data	Dynamic linked lists; grows on demand

4.1 C++ Memory Allocation Model: Dynamic Array of `std::list`

To build a graph supporting *V* vertices determined at runtime, C++ programs allocate a dynamic array of std::list<int> containers on the heap using pointers: std::list<int>* l = new std::list<int>[V];. This creates an array of size *V* where each index l[i] holds a doubly linked list storing neighbor indices.

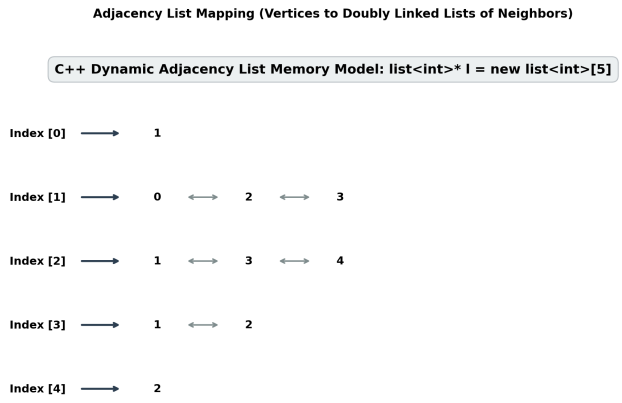


Figure 2: Architectural diagram of C++ Dynamic Array of Linked Lists (Adjacency List Memory Model).

5. Complete Object-Oriented C++ Graph Implementation

The production-ready C++ code below constructs an undirected graph using an Adjacency List, featuring dynamic heap allocation, destructor cleanup to prevent memory leaks, edge insertion, and complete adjacency display.

5.1 C++ Source Code (`Graph` Class)

```
#include
#include

using namespace std;

class Graph {
private:
    int V;                  // Total number of vertices
    list* l;          // Pointer to dynamic array of neighbor lists

public:
    // Constructor: Dynamically allocates memory for V adjacency lists
    Graph(int vertices) {
        this->V = vertices;
        this->l = new list[V];
    }

    // Destructor: Deallocates memory to prevent resource leaks
    ~Graph() {
        delete[] l;
    }

    // Method to add an edge between vertex u and vertex v
    void addEdge(int u, int v, bool isDirected = false) {
        l[u].push_back(v);
        if (!isDirected) {
            l[v].push_back(u); // Bidirectional insertion
        }
    }

    // Method to display the internal adjacency list
    void printAdjList() const {
        cout << "\n================ GRAPH ADJACENCY LIST ================\n";
        for (int i = 0; i < V; i++) {
            cout << "Vertex [" << i << "] -> ";
            for (int neighbor : l[i]) {
                cout << neighbor << " ";
            }
            cout << "\n";
        }
        cout << "======================================================\n";
    }
};

int main() {
    Graph g(5); // Construct graph with V = 5 vertices (0 to 4)

    // Insert edges matching sample structure
    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 3);
    g.addEdge(2, 4);

    g.printAdjList();
    return 0;
}
```

5.2 Executed Sample Graph Structure & Terminal Output

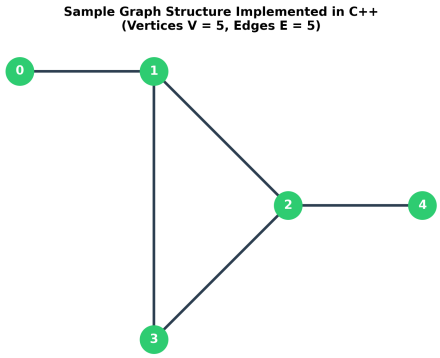


Figure 3: Sample 5-Vertex Undirected Graph used in C++ Execution Trace.

Terminal Execution Trace:

```
================ GRAPH ADJACENCY LIST ================
Vertex [0] -> 1
Vertex [1] -> 0 2 3
Vertex [2] -> 1 3 4
Vertex [3] -> 1 2
Vertex [4] -> 2
======================================================
```

6. Graph Traversal Paradigms: BFS & DFS Foundations

Systematic graph traversal is essential for solving search, connectivity, and shortest path problems.

6.1 Breadth-First Search (BFS)

- **Traversal Order:** Level-by-level exploration of degree-1 neighbors before degree-2 neighbors.
- **Primary Data Structure:** First-In-First-Out (FIFO) queue (std::queue<int>).
- **Key Application:** Finding shortest path in unweighted networks.

6.2 Depth-First Search (DFS)

- **Traversal Order:** Deep exploration along each branch before backtracking.
- **Primary Data Structure:** Last-In-First-Out (LIFO) call stack via recursion or std::stack<int>.
- **Key Application:** Cycle detection, topological sorting, connected components.

6.3 Traversal Complexity Summary

- **Time Complexity: $O(V + E)$** (Every vertex and edge is processed constant times).
- **Space Complexity: $O(V)$** (Required for visited tracking array and queue/stack).

7. Technical Interview Summary & Key Takeaways

1. **Non-Hierarchical Networks:** Graphs model generalized connections $G = (V, E)$ without root restrictions.
2. **Optimal Storage:** Always prefer Adjacency Lists ($O(V+E)$) over Adjacency Matrices ($O(V^2)$) for sparse graphs.

3. **C++ Memory Hygiene:** When dynamically allocating new std::list<int>[V], enforce destructor cleanup with delete[] l.
4. **Undirected Inserters:** Undirected graphs require dual neighbor updates: l[u].push_back(v) and l[v].push_back(u).